Computer science (AS)

Paper 1

1. Information representation
2. Communications and networking technologies
3. Hardware
4. Logic gates
5. Processer fundamentals
6. Assembly language programming
7. Monitoring and controlling systems
8. System software
9. Security
10. Ethics and ownership
11. Databases

Paper 2

12. Algorithm and design and problem solving
13. Data types and structures
14. Programming and data representation
15. Software development

1. Information representation

Number systems:

Denary – A base-10 number system that uses values 0-9

Binary – A base-2 number system that uses values 0 and 1

Hexadecimal – A base-16 number system that uses 16 symbols as shown below
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F)

Denary and hexadecimal both must be converted into binary values to be processed by computers.

Bit – a digit in the binary number system written using either of the symbols 0 and 1

Byte – A group of eight bits treated as a single unit

Nibble – A group of four bits (This can be represented by a singular hexadecimal value)

Binary to denary:

Take your binary value and above each digit starting from the right going to the left double the number starting from 1.

Q = Convert the Binary value 10010110 into Denary

Denary	128	64	32	16	8	4	2	1
Binary	1	0	0	1	0	1	1	0

Once complete take all the number above the digit 1 and add them all together

2 + 4 + 16 + 128 = 150

A = the denary value of 10010110 is 150

Denary to binary:

Put your binary value into a table and divide it by 2 each time and take the remainder aside

Q = what is the binary value of 150

2	150	0
2	75	1
2	37	1
2	18	0
2	9	1
2	4	0
2	2	0
2	1	1
	0	

A = the binary value of 150 is 10010110

When writing the binary value start from the bottom of the table

If there are not 8 digits in the 8 bit binary value place 0's at the bottom of the binary value in the table so at the start of the binary value.

Hexadecimal to denary/Denary to hexadecimal:

Just remember the pattern bellow

Hex	Den
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
1	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Numbers and quantities:

Decimal prefix – A prefix to define the magnitude of a value. Examples are kilo, mega, giga and tera respecting factors of 10^3, 10^6, 10^9 and 10^{12} respectively.

Binary prefix – A prefix to define the magnitude of a value. Examples are kibi, mebi, gibi and tebi representing factors of 2^{10}, 2^{20}, 2^{30} and 2^{40} respectively.

Internal coding of numbers:

One's compliment – The binary number obtained by inverting each digit in a binary number

Two's compliment – The one's complement of a binary number, plus 1. Used to represent negative numbers (better as it has one representation of 0)

Binary addition:

Rule 1 is 0 + 0 = 0
Rule 2 is 1 + 0 = 1
Rule 3 is 1 + 1 = 10 (the 1 is carried over)
Rule 4 is 1 + 1 + 1 = 11 (the 1 is carried over)

e.g.

Q = what is 10010100 + 00011110

	1	0	0	1	0	1	0	0
+								
	0	0	0	1	1	1	1	0
	1	0	1	1	0	0	1	0

A = 10110010

Overflow error – A type of error that occurs when a number larger that a resister can store is generated

	1	0	0	1	0	1	0	0
+								
	1	0	0	1	1	1	1	0
1	0	0	1	1	0	0	1	0

Overflow error

Binary coded decimal (BCD):

Storage of a binary value representing one denary digit in a nibble.

Packed BCD – When two BCD nibbles are stored in one byte

The BCD code uses a nibble to represent a denary digit. We consider the simple scheme where the digits are coded as the binary values from 0000 to 1001 being the denary values 0 to 9.

One BCD digit per byte:

00001000	00000101	00000000	00000011

Two BCD digits per bite:

10000101	00000011

Internal coding of text:

ASCII code – This is the binary code that the computer uses to represent characters. The 7-bit version has been standardised. It is stored on 1 byte with the most significant bit set to 0 and the remaining bits representing the character code.

Unicode – ASCII can only be used to represent so many characters as it is a 7-bit binary value. For this reason Unicode has been implemented. Unicode has 2 byes (16bits) but the number of codes available is determined by the number if bits that are not predefined.

Images:

Vector graphic – A graphic consisting of drawing objects defined in a drawing list.

- **Drawing object –** A component defined by geometric formulae and associated properties.
- **Drawing list –** Contains one set of values for each drawing object
- **Property –** Defines one aspect of the appearance of the drawing object.

Bitmaps:

- **Picture element (pixel) –** The smallest identifiable component off a bitmap image, defined by just two properties: its position on the bitmap matrix and is colour.
- **Colour depth –** The number of bits used to represent one pixel.
- **Bit depth –** The number of bits used to represent each of the RGB colours.
- **Image resolution –** The number of pixels in the bitmap file defined as the product of the width and the highest values.
- **Screen resolution –** The product of width and height values for the number of pixels that the screen can display.

A bitmap file does not define the physical size of a pixel or of the whole image. When the image is scaled the number of pixels in it does not change.

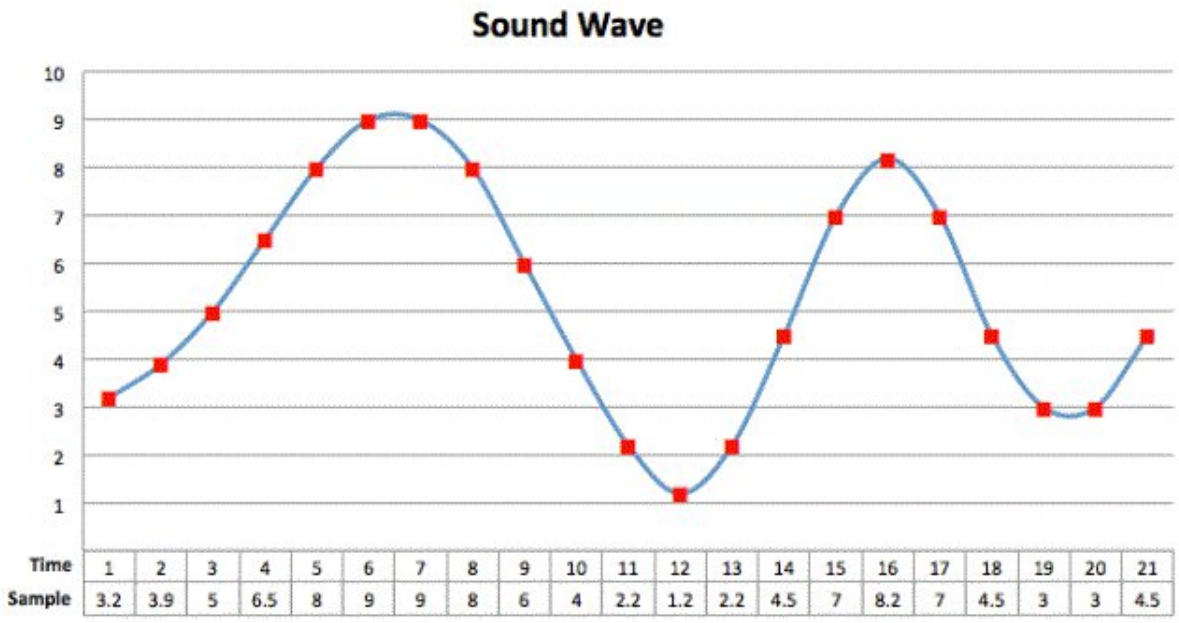
- **File header –** A set of bytes at the beginning of a bitmap file which the file contains information about the coding used.

Sound:

Sound is made up of sound waves. When sound is recorded, this is done at set time intervals. This process is knows as sound sampling.

Analogue data – Data obtained measurement of a physical property which can have any value from a continuous range of values.

Digital data – Data that has been stored as a binary value which can have one of a discreet range of values.



Each value is converted to binary to be processed by the computer. Each sound value is then used to playback the sound, when the recording is played.

Sampling – Taking measurements at regular intervals and storing the value

Sample rate – The number of samples recorded each second. (Measured in hertz) 1 hertz is equal to 1 sample per second. A common sample rate is 44.1 kHz (kilohertz), this would require 44100 samples to be taken each second.

Further data is required to represent sound, and this is sample resolution.

Sample resolution – The number of bits that are used to recode each sound sample.

Measuring data storage:

Data is measured in different units as shown below.

Unit	Description
Bit	This is the smallest measurement for data
Nibble	There are 4 bits in a nibble
Byte	There are 8 bits in a byte
Kilibyte (KiB)	There are 1024 bytes in a Kibibyte
Mebibyte (MiB)	There are 1024 Kibibytes in a Mebibyte
Gigibyte (GiB)	There are 1024 Mebibytes in a Gibibyte
Tebibyte (TiB)	There are 1024 Gibibytes in a Tebibyte
Pebibyte (PiB)	There are 1024 Tebibytes in a Pebibyte
Exbibyte (EiB)	There are 1024 Pebibytes in an Exbibyte

Most computers will do this for you but you must prove that you can do it manually in the exam for things like an image file and a sound file.

Data compression:

Compression – A method that uses an algorithm to reduce the size of the file.

Compressing a file will reduce the size of the file that can have several benefits, such as:

- Not as much storage space is need to store the file.
- It will take less time to transmit the file from one device to another.
- It will be quicker to upload and download the file.
- Not as much bandwidth is needed to transmit the file over the internet.

There are 2 types of compression:

- Lossy compression – Coding techniques that cause some information to be lost so that the exact original file cannot be recovered in subsequent decoding.
- Lossless – Coding techniques that allow subsequent decoding to recreate exactly the original file.

2. Communication and networking technologies

Wide area network (WAN) – A network connecting computers on different sites, possible thousands of kilometres apart

The benefits of having computers connected be a WAN are:

- A ‘job’ could be run on a remote computer that had the required application software

A data archive that was stored on a remote computer could be accessed

- A message could b be transmitted electronically to a user on a remote computer.

Local area network (LAN) – A network connecting computers in a single room, in a single building got no a single site.

The benefits of connecting PCs in a LAN include the following:

- The expense of installing application software in each individual PC could be saved by installing the software on an application server attached to the LAN instead.
- A file server could be attached to the LAN that allowed users to store lather files and also allowed files to be shared between users.
- Instead of supplying individual printers to be connected to a user’s PC, one or more printers could be attached to a print server that was connected to the LAN; these could be higher quality prints
- Managers in organisations could use electronic mail to communicate with staff rather than sending round memos on paper
- The ‘paper-less office’ became a possibility, where files were to be stored in digital form on a file server rather than as paper copies in a filing cabinet

The client-server model:

Client-server – An architecture where a client runs a supplication provided ne a server on a network

Thin-client – A client that only provides input and receives output from the application.

- Chooses an application to run on the server
- Sends output data to the server when requested by the application
- Receives output from the application

Thick-client – A client that carries out at least some of the processing itself

- Chooses an application provided by the server
- Possibly carries out some processing before running the application on the server and also after receiving output from the application
- Alternatively, possible downloads the application from the server and runs the application itself

The client-server approach is the choice in the following circumstances:

- The server stores a database which is accessed from the client system
- The server stored a web application which allows the client system to find or, sometimes, supply information
- The server stores a web application which allows the client system to carry out an e-commerce or financial transaction

File sharing:

If a user uploads files to a file server then the client-server operation can be used by another user to download these from the server. You can use the client server operation or alternatively use peer-to-peer networking

The peer-to-peer network has the following advantages:

- It avoids the possibility of congestion on the network when many clients are simultaneously attempting to download files
- Parts of a file can be downloaded separately
- The parts are available from more than one host

The client-server model has the following advantages:

- It allows the organisation to control the downloading and use of files
- The files can be better protected from malware attacks because the files are stored in one server which will be regularly scanned using appropriate anti-virus software

Network topologies:

Different mods data can be sent:

- Simplex – Data flows one way
- Half duplex – Data flows both ways but only one at a time
- Full duplex – Data flows both ways at the same time

A 'message' is any type of data, which can be sent as either:

- A broadcast – One-to-all communication
- A multicast – One source to many destinations
- A unicast – One-to-one communication

Topology – The configuration of a network that defines how the various devices on the network are connected. There are 3 types of topologies:

- Bus topology – Contains one shared link to which all devices are attached
- Mesh topology – Contains direct links between devices
- Star topology – Each end-system is linked to a central device

End-system – A computer or service connected to a network

Hybrid network – A collection of connected LANs where some of them have different topologies or supporting technologies.

Transmission media:

Cable – A transition using copper wire or fibre-optic

Wireless – A transmission using radio, microwave or infrared

Bandwidth – A measure of the amount of data that can be transmitted per second

LAN hardware:

Server – A system providing a service to end-systems

Repeater – A device that connects two cable sand provides a full-strength signal to the second cable

Bridge – A device that connects two segments of a LAN

Network Interface Card (NIC) – A component used to identify the end-system

Switch – A connecting device that can send a unicast message

Wireless Access point (WAP) – The connecting device in a WiFi LAN

Wireless Network Interface Card (WNIC) – Provides the NIC function in a WiFi LAN

Ethernet:

Widely used wired medium for data transmission in LANs and WANs

Used mostly in star and bus topology

Collisions are resolved by the CSMA/CD (Carrier Sense Multiple Access with Collision Detection) method:

- Monitors channel and sends only when line is idle
- When channel is busy, it waits a random time and retransmits
- While transmission, it listen for other transmission
- After a collision, transmission is aborted and both nodes retry after random times

The internet infrastructure:

To describe the Internet as a WAN pays a little attention to its size and complexity. The internet is the biggest internetwork in existence. Furthermore, it has never been designed as a single 'whole'; it has just evolved to reach its current form and is still evolving towards whatever future form it will take.

Internet service provider (ISP) – Gives internet access to an individual or company. This function is now performed by what we call an 'access ISP'. These access ISPs then connect to what we call 'middle tier' or regional ISPs, which in turn are connected to tier 1 (or 'backbone') ISPs. An ISP is a network and connections between ISPs are handled by Internet Exchange Points (IXPs). The tier 1 ISPs are at the top of the hierarchy, along with major Internet content providers.

Router – A device that acts as a node on the internet

Public switched telephone network (PSTN) – During the early years of networking the telephone network carried along analogue voice data. However, digital data could be transmitted provided that a modem was used to convert the digital data to analogue signals.

Cell phone networks – For users of devices with mobile (cell) phone capability there is an alternative method for gaining Internet access. This is provided by mobile phone companies acting as ISPs. The mobile phone, equipped with the appropriate software, communicates with a standard cell tower to access the wireless telephone network, which in turn provides a connection to the Internet.

Applications that make use of the internet:

The World Wide Web (WWW) – The web consists of an enormous collection of websites each having one or more web pages. The special feature of a web page of that it can contain hyperlinks which, when clicked, give direct and essentially immediate access to other web pages.

Cloud computing – Cloud computing is the provision of computing services usually via the Internet

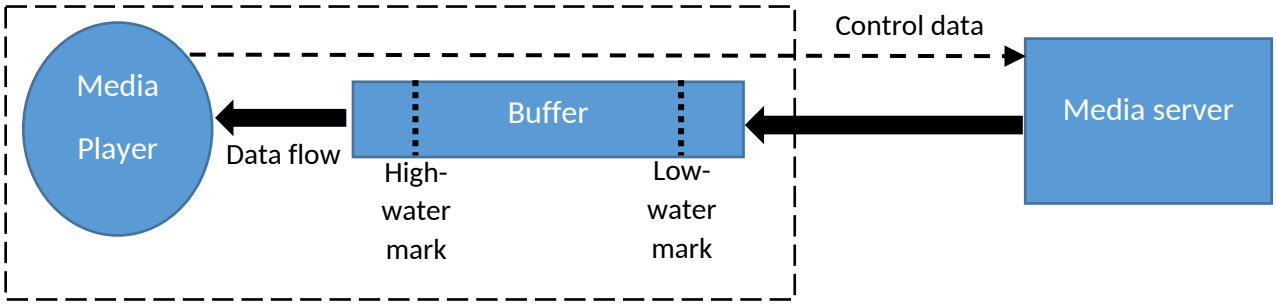
- Private cloud – Owned by and only accessed by an organisation
- Public cloud – Owned by a service provider for general access

Bit streaming – Before data is transmitted it is stored on bytes which can be transmitted one after the other as a byte stream.

- On-demand – When the bit stream content is transmitted at a time chosen by the user
- Real time – When the bit steam is transmitted as it is produced

Bit rate – The number of bits transmitted per second

Buffer – The buffer must deliver data to the user at the correct bit rate for the media being used. Data that is sent to the buffer should be sent at a higher rate to allow for unexpected delays.



IP addressing:

IPv4 – A 32-bit long, hierarchical address of a device on the internet

Class	Class identifier	Number of bits for netID	Number of bits for hostID
Class A	0	7	24
Class B	10	14	16
Class C	110	21	8

Each byte is written in its denary equivalent separated by dots:

10000000 00001100 00000010 00011110

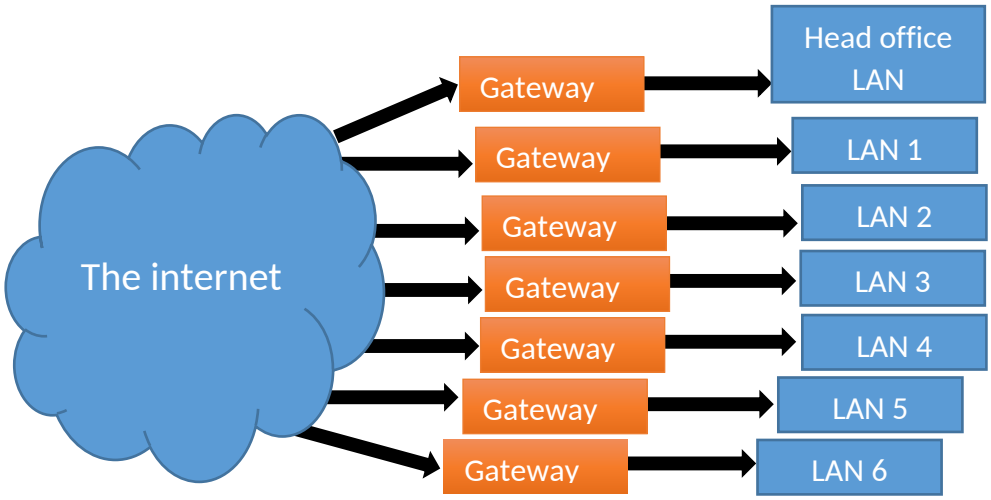
This is written in dotted decimal notation as:

128.12.2.30

Classless inter-domain routing (CIDR) – This retains the concept of a netID and a hostID but removes the rigid structure and allows the split between the netID and the hostID to be varied to suit individual need. To achieve this we add an 8-bit suffix to the address that that specifies the number of bits for the netID. If we for example define the suffix as 21, that means that 21 bits are used for the netID and there are 11 bits remaining (of 32-bits) to specify hostIDs allowing 2^{11} hosts.

110000110000110000000011000001110/00010101
netID suffix

Sub-netting – Let's say we have 150 employees with their own computer workstation. There are also 6 individual department LANs and one head-office LAN.

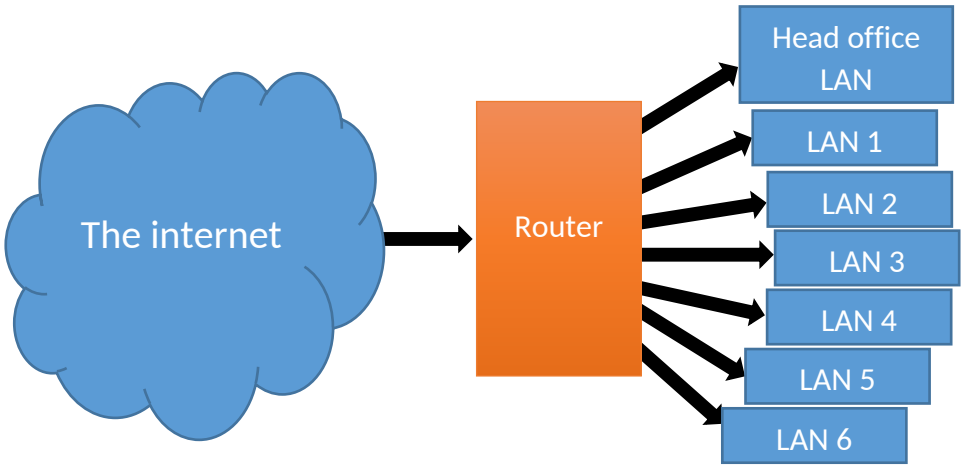


The organisation would need seven individual Class C netIDs; one for each LAN. Each of these would point to one of the LAN gateways. The netID for each LAN would be identified by the first 24 bits of the IPv4 address, leaving 8 bits for the host ID. This would mean 256 individual codes for identifying different workstations on just one LAN. For the seven LANs

the total number of workstations that could be identified would be 1792 (256 * 7). This would mean that there are 1642 unused addresses unavailable to any other organisation.

The sub-netting solution for this organisation would require allocation just one Class C netID. For example, the IP addresses allocated might be 194.10.9.0 to 194.10.9.255 where the netID compromises the first three bytes, represented by the decimal values 194, 10 and 9.

The sub-netting works by having a defined structure for the 256 codes constituting the hostID. A sensible solution for this organisation is to use the top three bits as a code for the individual LANs and the remaining five bits as codes for the individual workstations.



On the internet all the allocated IP addresses have a netID pointing to the router. The router then has to interpret the hostID to direct the transmission to the appropriate workstation on one of the LANs via a gateway. Examples of workstation identification:

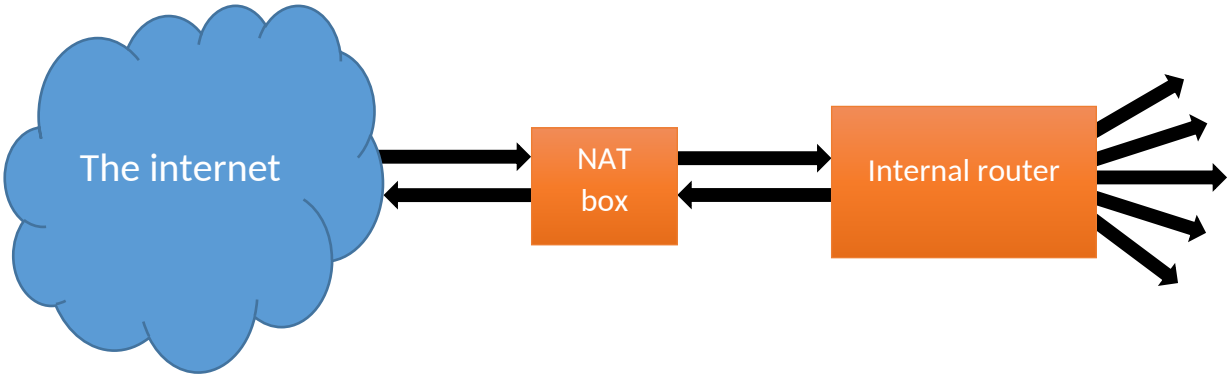
- HostID code 00001110 would be the address for the workstation 14 on the head office LAN 0 (LAN 000)
- HostID code 01110000 would be the address for workstation 14 on the 16 on LAN 3 (LAN 011)

With 150 workstations the organisation hasn't used all of the 256 allocated IP addresses. However, there are only 106 unused which is a reasonable number to have available in case of further expansion. Only the netID has been used leaving the other 6 the might have been used still available for other organisations to use.

Network address translation (NAT):

The final scheme to be considered is different in that it deviates from the principle that every IP address should be unique. In this scheme. Provision has been made for large organisations to have private networks (intranets) which use the same protocols as those used for the Internet. One justification for using a private network has always been that this provides extra security because of the isolation from the Internet, However, this is no longer normal practice. Organisations want private networks but they also want Internet connectivity.

The solution for dealing with the addressing is to use network address translation (NAT)



The NAT box has one IP address which is visible over the internet and so can be used as a sending address or as a receiving address. Internally the IP addresses have to be chosen form one of the three ranges of IP addresses shown below that have been allocated for such networks. (Don't try remember these numbers)

Lower bound	Upper bound
10.0.0.0	10.255.255.255
172.16.0.0	172.31.255.255.
192.168.0.0	192.168.255.255

The important point is that each address can be simultaneously used by any number of different private networks. There is no knowledge of such use on the Internet itself or in any other private network. The interface in the NAT box has software installed to examine each incoming or outgoing transmission. There can be a security check before an incoming transmission is directed to the correct internal address. The diagram shows undefined arrows from the router connected to the NAT box. These indicate that the network structure within the organisation could take many different forms.

Static and dynamic IP addresses:

Static address – This never changes and can be provided of a user is prepared to pay an extra charge

Dynamic address – This is one that the ISP is free to change if it suits but more importantly the address is available for re-allocation once a user disconnects from the Internet

IPv6 addressing:

This uses a 128-bit bit addressing scheme allowing 2^{128} different addresses. The addresses are written in a colon hexadecimal notation. The code is broken into 16-bit parts, with each part represented by four hexadecimal characters (some abbreviations are allowed)

Domain names:

For a user to access a web page it is impractical to identify each IP address by its dotted decimal value. To solve this we use a domain name service (DNS)

Domain name service (DNS) – A hierarchical distributed database installed on domain name servers that are responsible for mapping a domain name to an IP address. Also known as domain name system.

Looking up a domain name to find an IP address is called 'name resolution'. For such a query there are three possible outcomes.

- If the domain is under the control of the server to which the query is sent then an authoritative and correct IP address is returned
- If the domain is not under the control of the server, an IP address can still be returned if it is stored in a cache of recently requested addresses but it might be out of date
- If the domain in the query I remote then the query is sent to a root server which can provide an address for the name server of the appropriate top-level domain. This in turn can provide the address for the name server in the next lower domain. This continues until the query reaches a name server that can provide an authoritative IP address

3. Hardware

Computer system hardware functionality:

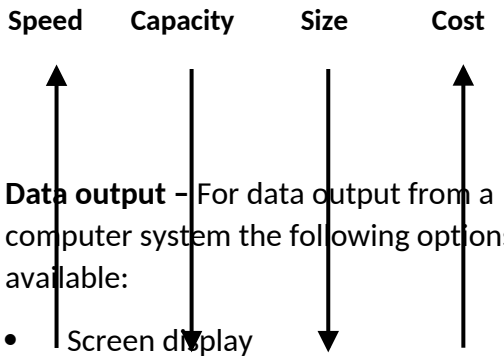
A computer system has to support three major areas of operational capability:

- The processing of data
- The storage of data
- The input and output of data

At the heart of the system the processing of data is carried out by the CPU (Central Processing Unit). A general-purpose computer system comprises a processor, memory, and I/O functionality.

Data storage:

Component	Category
Register	Processor component
Cache memory	Primary storage
Main memory	
Hard disk	Secondary storage
Auxiliary storage	



Data output – For data output from a computer system the following options are available:

- Screen display
- Hardcopy using a printer or plotter
- Virtual headset display
- A speaker
- Writing any of the data storage devices described earlier
- Transmission on a network link

Data input – For the input of data to a computer system the following are among the options available:

- Keyboard or keypad entry by a user
- A user using a game controller
- A user using a scanner
- A user using a microphone in tandem with voice recognition software
- Reading form any of the storage devices described earlier
- Transmission on a network link

Note the any input or output in a computer system are controlled by an I/O sub-system.

Embedded systems:

Miniature computer systems such as microprocessors that are often a part of a more extensive system. Each embedded system performs a few specific functions.

Memory components:

Read-only memory (ROM) – non-volatile memory that cannot be written to but can be read from any number of times.

Random access memory (RAM) – Volatile memory that can be read from or written to any number of times. There are 2 types of RAM:

SRAM	DRAM
Doesn't need to refresh; hence, it uses less power and faster access time	Has to be refreshed; it has slower access times and needs higher power
More complex circuitry, hence more expensive	Only a single transistor & capacitor, hence less expensive to purchase
Each bit is stored in a flip-flop	Each bit is stored as a charge
Has lower data density	Has higher data density
Used in cache memory	Used in main memory

Buffer – A temporary storage created for data transmitted from one part of the system to another which functions as a queue.

Secondary storage devices:

Magnetic media – Magnetic media is any storage medium that uses magnetic patterns to represent information. Common examples include hard drives, floppy disks, and magnetic tapes

Optical media – Optical media is a type of data storage that uses light to read or write data to an underlying optical media. This means that a laser is used to read and write data on the media. Common examples include compact discs (CDs) and digital versatile discs (DVDs)

Solid state media – Solid-state media is a type of storage that uses electronic circuits to store data persistently. It has no moving parts. This means that solid-state media is much faster, more durable, and more energy-efficient than traditional hard drives. Solid-state drives (SSDs) are a common example of solid-state media.

Output device provided for a user of a general-purpose computer system:

Screen display – Screen displays use pixels, which are made up of three sub-pixels (red, green, blue) that emit varying levels of light to create a full range of colours. Traditionally, screens used phosphors that glowed when hit by electrons. Modern screens use liquid crystal displays (LCDs), which are backlit by LEDs. Each pixel cell contains liquid crystals that can be manipulated by voltage to change the polarization of light, affecting what is displayed on the screen.

- Liquid-crystal display (LCD) – A screen back-lit by light-emitting diodes and with liquid crystal cells sandwiched between polarisers.

Virtual reality headset – Virtual headsets consist of 2 lenses, (an LCD) display, a circuit board with sensors, a cover and foam padding. The display provides a simulation of a 3D environment generated by a 3D graphics package. The user can 'move' in the virtual environment by moving their head or using controllers

Hard-copy output of text – Two technologies have dominated the printing of documents from data stored in a computer system.

- Laser printer – Laser printers use a laser beam and rotating mirrors to create an image on a photosensitive drum. This image is then transferred to the paper using toner, which is attracted to the charged image. Heat is applied to fuse the toner to the paper.

- Inkjet printers – Inkjet printers work by spraying tiny droplets of ink onto paper. They use a print head with tiny nozzles that are heated to create bubbles of ink. These bubbles are then forced out of the nozzle, creating a fine mist of ink droplets.

Hard-copy graphics output – Specialised technical applications often require a more accurate representation to be created on paper. This required the use of a graphic plotter. A plotter uses pens to write, usually, on a large sheet of paper constrained by sprockets along one pair of sides. The sprockets can move the paper forwards or backwards and pens can either be parked or in use at any given time. The controlling circuitry and software can create the drawing directly from the original vector graphic file.

A 3D design is created in a suitable computer-aided design (CAD) package. The design is split into layers. The data for the first layer is transmitted to the 3D printer. Rather than using ink to draw the layer, the 3D printer uses a nozzle to squirt material on to the printer bed to create a physical layer to match the design. This process is repeated for successive layers. When the whole object has been formed it has to be curved in some way to ensure that the layers are stick together and the material has been converted to the form required for the finished product.

Input devices provided for a user of a general-purpose computer system:

Keyboard – The keyboard allows a user to input text data.

- The keys are positioned above a key matrix, which consists of a set of rows and wires and another set of columns of wires.
- Pressing a key cause's contact at one of the points where wires cross.
- The microprocessor continuously tests to see of any electrical circuit involving a row wire and a column wire has become closed.
- When the microprocessor recognises that a circuit has become closed, it can identify the particular intersection that is causing this.
- The processor then uses data stores in the ROM to identify the character code relating to the key associated with that intersection and sends this character to the screen.

The screen – There are a number of ways in which a user can cause in which a user can cause data to be input through an interaction with a screen. A graphical user interfaces (GUI) provides a number of different types of screen icon, each of which allows the user to control data input. The user needs a pointing mechanism to use a GUI effectively. One example of a pointing mechanism is a computer mouse that controls the position of a cursor on the screen. The screen is now not just an output device but also an input device activated by a mouse click.

Touch screen – There are 2 types of touch screens:

- Resistive touch screen – A flexible surface that causes contact between electrically resistive layers beneath when touched.
- Capacitive touch screen – A rigid structure above a conductive layer that undergoes a change in electrical state when a finger touches the screen.

Input of a graphic – Methods of Image Capture:

- Webcam: Streams live video directly into a computer.
- Digital Camera: Captures images and videos that can be downloaded to a computer.
- Scanner: Creates a digital representation of a physical image (like a document or photograph) by scanning it with a light source.

Input and output of sound:

Voice input and output – IP telephony and video conference are two applications that require both voice input and voice output. Voice recognition can be used as an alternative way for data input.

- Input – For input a microphone is needed. This is a device that has a diaphragm, a flexible material that is caused to vibrate by an incoming sound. The diaphragm is connected to a circuitry that change the electrical signal due to vibrations. An analogue-to-digital converter (ADC) is used so that the signal can be processed by the computer.
- Output – For output a speaker is needed. A digital-to-analogue converter (DAC) is used. A current is sent through a coil suspended within the magnetic field provided by a permanent magnet in the speaker. As the current changes the coil moves backwards and forwards. This movement controls the movement of a diaphragm, which causes sound to be created.

The input and output are controlled by a sound (audio) card installed in the computer.

4. Logic gates

Boolean logic and problem statements:

Logic proposition – A statement that is either TRUE or FLASE

- Colombo is further north than Singapore

Problem statement – An informal definition of an outcome which is dependent on one logic proposition or a combination of two or more logic propositions.

- The air-conditioning system is set to come on in an office only during working hours but also only if the temperature is above $20^{O}C$

Boolean operators:

The problem statement identified above can be more formally expressed in a form that is suitable for handling with Boolean logic. To do this it is necessary to use Boolean operators. The three basic Boolean operators are AND, OR and NOT.

The definition for AND, OR and NOT can be expressed as:

- A AND B is TRUE if A is TRUE and B is TRUE
- A OR B is TRUE if A is TRUE or B is TRUE
- NOT A is TRUE if A is FALSE

Each original problem statement has now been rephrased to include a form of logic expression.

Logic expression – Logic propositions combined using Boolean operators, which may be equated to a defined outcome.

Any logic expression can be constructed using only the Boolean operators AND, OR and NOT but it is often convenient to use other operators. Here are the definitions for the three other operators that you should be familiar with:

- A NAND B is TRUE if A is FASLE and B is FALSE
- A NOR B is TRUE if A is FALSE and B is FALSE
- A XOR B is TRUE if A is TRUE or B is TRUE but not both of them

Truth tables:

A truth table is a way to represent any logic expression or describing the possible outputs from a logic circuit.

In a truth table TRUE is represented as 1 and FALSE is represented by 0

This is a truth table for an AND gate

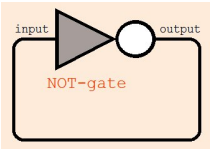
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

Logic circuits and logic gates:

The digital circuits that constitute the inner workings of a computer system operate as logic circuits where each individual part of the circuit is either in an on state (1) or an off state (0)

Logic gate – A component of a circuit that has an operation matching that of a Boolean operator

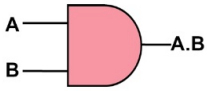
Here are all the types of logic gates:

1. NOT gate 

- If the input is a 1 the output will be a zero and vice versa.

A	X
1	0
0	1

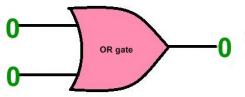
The logical expression – X = NOT A

2. AND gate 

- It is only high when the inputs are high. It has at least 2 inputs.

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

The logical expression – X = A AND B

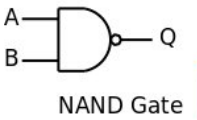
3. OR gate 

The OR gate has at least 2 inputs and the output is high when at least one input is high. The output is only 0 when both inputs are zero.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

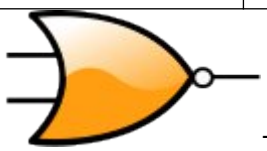
X = A OR B

X = A + B

4. NAND gate 

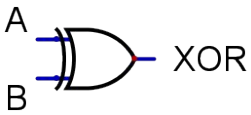
– The output is only low when both inputs are high. This is the opposite of the AND gate.

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

5. NOR gate 
– This is the opposite of the OR gate. This is only high when both inputs are low.

A	B	X
0	0	1

0	1	0
1	0	0
1	1	0

6. XOR gate 

- The output is only high when the inputs are different.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

5. Processer fundamentals

The von Neumann model of a computer system:

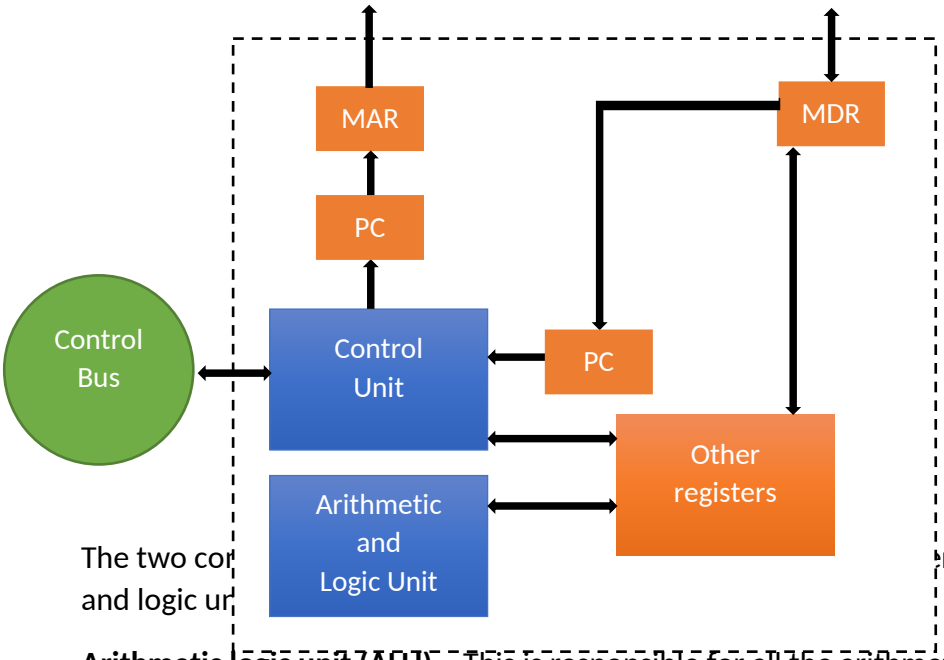
John von Neumann was the first person to describe the basic principles of a computer system and its architecture in a publication.

The model von Neumann described has the following basic features:

- There is the processer – the central processing unit (CPU)
- The processor has direct access to memory
- The memory contains a 'stored program' (which can be replaced by another at any time) and the data required by the program
- The stored program consists of individual instruction
- The processer executes instructions sequentially

Central processing unit (CPU) architecture:

Simplified schematic diagram of a processer:



The two cor rations are the arithmetic and logic ur

Arithmetic logic unit (ALU) – This is responsible for all the arithmetic or logic processing requirements of the instructions in a running program.

Control unit – It controls the flow of data throughout the processer and the rest of the whole computer system. It also ensures that program instructions are handled correctly. A vital part of the control unit is a clock that is used by the unit to synchronise processes. There are 2 types of clocks:

- Internal clock – Controls the cycles of activity within the processer
- System clock – Controls the cycles of activity outside the processer

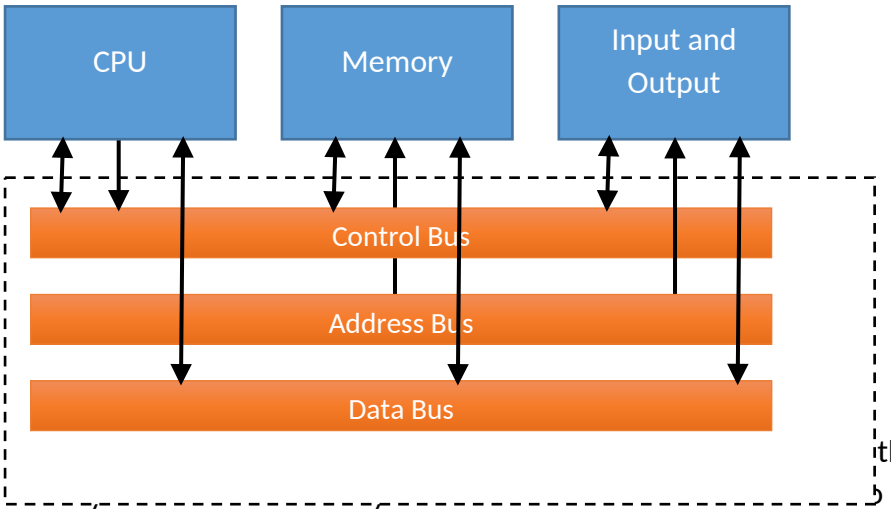
Registers – These are storage components which, because they are placed very close to the ALU, allow very short access times. Each register has limited storage capacity, typically 16, 32 or 64 bits:

Accumulator – A general-purpose register that stores a value before and after the execution of an instruction by the ALU.

Register name	Abbreviation	Register's function
Current instruction register	CIR	Stores the current instruction while it is being decoded and executed
Index register	IX	Stores a value; only used for indexed addressing
Memory address register	MAR	Stores the address of a memory location or an I/O component which is about to have a value read from or written to
Memory data register (memory buffer register)	MDR (MBR)	Stores the data that has just been read from memory or is just about to be written to memory

Program counter	PC	Stores the address of where the next instruction is to be read from
Status register	SR	Contains buts that are wither set or cleared which can be referenced individually.

The system bus:



the memory controller to it can be to the I/O system to identify the source or destination of the data.

Data bus – A component that can carry data from the processor to the memory or to an output device or can carry from the memory or from an input device.

Control bus – This is a bidirectional bus which transmits a signal from the control unit to any other system component or transmits a signal to the control unit.

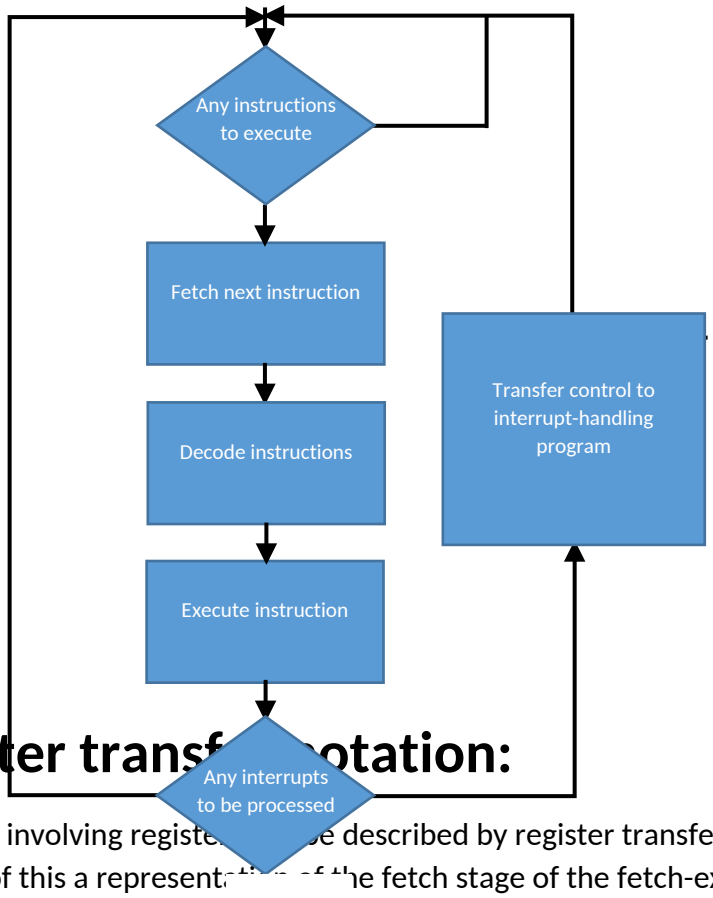
Factors contribution to system performance:

- Clock speed – One clock cycle defines the shortest possible time that any action can take.
- Cores – Each core is a separate processer. Performance improves with increasing number of cores.
- Cache memory – Performance improves with increased storage size for the cache and increased rate of access.
- Bus width – The bus width defines the number of bits in the address's binary code

Word – A small number of bytes that can be handled as a unit by the computer system.

A typical word lengths are 16, 32 or 64 bits. Word length will influence the system architecture design in regard to the capacity of the components.

The fetch, decode and execute cycle:



Register transfer notation:

Operation involving registers can be described by register transfer notation. A simple example of this a representation of the fetch stage of the fetch-execute cycle.

MAR ß [PC]

PC ß [PC] + 1; MDR ß [[MAR]]

CIR ß [MDR]

Interrupt handling:

There are many different reasons for an interrupt to be generated. Some examples are:

- A fatal error in program
- A hardware fault
- A need for I/O processing to begin
- User interaction
- A timer signal

An interrupt can be handled by the following steps:

- The contents of the program counter and any other registers are stored somewhere safe in memory
- The appropriate interrupt Service Routine (ISR) program is initiated by loading its start address into the program counter

- When the ISR program has been executed there is an immediate check to see if further interrupts need handling
- Further interrupts are dealt with by repeated execution of the ISR program
- If there are no further interrupts, the safely stored contents of the restored to the CPU and the originally running program is resumed

6. Assembly language programing

Machine code instructions:

We need to start with a few facts:

- The only language that the CPU recognises is machine code
- Machine code consists of a sequence of instructions
- An instruction contains opcode
- An instruction may not have an operand but up to three operands are possible
- Different processers have different instruction sets associated with them
- Different processers will have comparable instructions for the same operations, but the coding of the instruction will be different

For a particular processer, the following must be defined for each individual machine code instruction

- The total number of bits or bytes for the whole instruction
- The number of bits that define the opcode
- The number of operands that are defined in the remaining bits
- Whether the opcode occupies the most significant of the or significant bits

Opcode – Defines the action associated with the instruction

Operand – Defines any data needed by the instruction

Machine code instructions – A binary code with a defined number of bits that comprises an opcode and, most often, one operand

The number of bits needed for the opcode depends on the number of different opcodes in the instruction set for the processor. The opcode can be structured with the first few bit defining the operation and the remaining bits associated with addressing. A sensible instruction format for our simple processor is shown below.

Opcode			
Operation	**Address mode**	**Register Addressing**	**Operand**
4 bits	2 bits	2 bits	16 bits

This has an 8 bit opcode consisting of four bits for the operation and the remaining two bits for addressing registers. This allows 16 different operations each with one of four addressing modes. This opcode will occupy the most significant bits in the instruction. Because in some circumstances the operand will be a memory address it is sensible to allocate 16 bits for it. This is in keeping with the 16-bit address bus.

When an instruction arrives in the CPU the control unit checks the opcode to see what action it defines. This first step in the decode stage of the fetch-execute cycle can be described using the register transfer notation. However, a slight amendment is needed to the format. The following shows the transfer of bits 16 to 23, which represent the opcode, from the current instruction register to the control unit:

CU <- [CIR (23:16)]

Assembly language:

Assembly language – A low-level language related to machine code where opcodes are written as mnemonics and there is a character representation for an operand.

Assembler – A program used to translate an assembly language program into machine code.

Mnemonic – A symbolic abbreviation.

For each machine code instruction there is an equivalent assembly language instruction which comprises:

- A mnemonic for the opcode
- A character representation

The use of an assembler allows a programmer to include some special features in an assembly language program for example:

- Comments
- Symbolic manes for constants

- Labels for addresses
- Macros
- Directives

Directives – An instruction to the assembler program.

Symbolic, relative and absolute addressing:

An assembly program using symbolic addressing with explanations:

Assembly language program using symbolic addressing	Explanation of each instruction
IN	A single number is input at the keyboard and its ASCII code is stored in the accumulator
SUB #48	This subtraction converts the ASCII code into the binary code for the number (see Task 6.01)
STO MAX	The number in the accumulator is stored at the address labelled MAX:
LDM #0	Loads zero into the accumulator
STO TOTAL	The zero in the accumulator is stored at the address labelled TOTAL:
STO COUNT	The zero in the accumulator is stored at the address labelled COUNT:
STRTLP:IN	A single number is input at the keyboard and its ASCII code is stored in the accumulator
SUB #48	This subtraction converts the ASCII code into the binary code for the number
ADD TOTAL	Adds the value at address labelled TOTAL: to the value in the accumulator and stores the sum in the accumulator
STO TOTAL	The number in the accumulator is stored at the address labelled TOTAL:
LDD COUNT	Loads the value stored at address COUNT: into the accumulator
INC ACC	Adds 1 to the value in the accumulator
CMP MAX	Compares the value in the accumulator with the value stored at address MAX:
JPN STRTLP	If the compared values are not equal the program jumps to the instruction labelled STRTLP:
END	The execution of the program has finished
MAX:	A labelled address where a value can be stored
TOTAL:	A labelled address where a value can be stored
COUNT:	A labelled address where a value can be stored

A simple assembly language program using relative and absolute addressing:

Assembly language program using relative addressing	Assembly language program using absolute addressing	
(0) IN	(200)	IN
(1) SUB #48	(201)	SUB #48
(2) STO [BR] + 15	(202)	STO 215
(3) LDM #0	(203)	LDM #0
(4) STO [BR] + 16	(204)	STO 216
(5) STO [BR] + 17	(205)	STO 217
(6) IN	(206)	IN
(7) SUB #48	(207)	SUB #48
(8) ADD [BR] + 16	(208)	ADD 216
(9) STO [BR] + 16	(209)	STO 216
(10) LDD [BR] + 17	(210)	LDD 217
(11) INC ACC	(211)	INC ACC
(12) CMP [BR] + 15	(212)	CMP 215
(13) JPN [BR] + 7	(213)	JPN 207
(14) END	(214)	END
(15)	(215)	
(16)	(216)	
(17)	(217)	

The assembly process for a two-pass assembler:

For any assembler there are a number of things that have to be done with the assembly language code before any translation can be done. Some examples are:

- Removal of comments
- Replacements of a macro name used in an instruction by the list of instructions that constitute the macro definition
- Removal and storage of directives to be acted upon later

Two-pass assembler – This translates assembly language code into machine code in two stages:

- First Pass: Collects label information and assigns addresses.
- Second Pass: Resolves labels to addresses and generates the final machine code.

This approach ensures that labels can be used before they are defined and that all addresses are correctly resolved.

The opcode is used for mnemonics and the oprand is used for the data.

Machine code		Assembly code	
Opcode	Operand		
0001 0000			IN
0110 0001 0000 0000 0011 0000			SUB #48
0100 0100 0000 0000 0000 1111			STO MAX
0010 0001 0000 0000 0000 0000			LDM #0
0100 0100 0000 0000 0001 0000			STO TOTAL
0100 0100 0000 0000 0001 0001			STO COUNT
0001 0000		STRTLP:	IN
0110 0001 0000 0000 0011 0000			SUB #48
0100 0101 0000 0000 0001 0000			ADD TOTAL
0100 0100 0000 0000 0001 0000			STO TOTAL
0010 0101 0000 0000 0001 0001			LDD COUNT
0101 0101			INC ACC
1000 0100 0000 0000 0000 1111			CMP MAX
1010 0100 0000 0000 0000 0110			JPN STRTLP
1111 1111			END
0000 0000		MAX:	
0000 0000		TOTAL:	
0000 0000		COUNT:	

Some points to not are as follows:

- Most of the instructions have an operand which is a 16-bit binary number.
- Usually this represents an address but for the SUB and LDM instructions the operand is used as a value.
- There is no operand for the IN and END instructions.
- The INC instruction is a special case.
- There is an operand in the assembly language code but this just identifies a register.
- In the machine code the register is identified within the opcode so no operand is needed.
- The machine code has been coded with the first instruction occupying address zero.
- This code is not executable in this form but it is valid output from the assembler.
- Changes will be needed for the addresses when the program is loaded into memory ready for it to be executed.
- Three memory locations following the program code have been allocated a value zero to ensure that they are available for use by the program when it is executed.

Addressing modes:

Addressing mode – When the instruction uses a value this defines how the operand must be used to find the value

Addressing mode	Use of the operand
Immediate	The operand is the value to be used in the instruction; `SUB #48` is an example.
Direct	The operand is the address which holds the value to be used in the instruction; `ADD TOTAL` is an example.
Indirect	The operand is an address that holds the address which has the value to be used in the instruction.
Indexed	The operand is an address to which must be added the value currently in the index register (IX) to get the address which holds the value to be used in the instruction.

Assembly language instructions:

Data movement:

Some instruction formats for data movement:

Instruction opcode	Instruction operand	Explanation
LDM	#n	Immediate addressing. Load the number n to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
LDD	<address>	Direct addressing. Load the contents at the given address to ACC.
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
MOV	<register>	Move the contents of the accumulator to the given register (IX).
STO	<address>	Store the contents of ACC at the given address.

Examples of some data stored in memory:

Memory address	Memory content
100	234
101	208
102	201
103	110
104	108
105	206
106	101
107	102
INDEXVALUE:	3

Accumulator

Index register

LDD 103
LDI 106
STO 106
LDD INDEXVALUE
MOV IX
LDX 102

Input and output – There are two instructions provided for input or output. In each case the instruction has only an opcode, there is no operand.

- The instruction with opcode **IN** is used to store in the ACC the ASCII value of a character typed keyboard.
- The instruction with opcode **OUT** is used to displayed on the screen the character for which the ASCII code is stored in the ACC

Comparisons and jumps:

Instruction opcode	Instruction operand	Explanation
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with the number n
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False

Arithmetic operations:

Instruction opcode	Instruction operand	Explanation
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
SUB	#n	Subtract the denary number n from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)

Shift operations:

There are two shift instruction available:

- LSL #n
  where the bits in the accumulator are shifted logically n places to the left
- LSR #n
  where the bits are shifted to the right

In a logical shift no consideration is given as to what the binary code in the accumulator represents. Because a shift operation moves a bit from the accumulator into the carry bit.

- Logical shift – Where bits in accumulator are shifted to the right or to the left and a zero moves into the bit position vacated.
- Cyclic shift – Similar to a logical shift but bits shifted from one end reappear at the other end.
- Arithmetic shift – Uses the shift to carry out multiplication or division of a signed integer stored in the accumulator.

Bitwise logic operation:

Instruction opcode	Instruction operand	Explanation
AND	#Bn	Bitwise AND operation of the contents of ACC with the binary number n
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#Bn	Bitwise XOR operation of the contents of ACC with the binary number n
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#Bn	Bitwise OR operation of the contents of ACC with the binary number n
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>

Further consideration of assembly language instructions:

Register transfer notation – We can use this to describe the execution of an instruction. For example, the *LDD* instruction is described by:

ACC <- [[CIR (15:0)]]

The instruction is in the CIR and only the 16-bit address needs to be examined to identify the location of the data in memory. The contents of that location are transferred into the accumulator.

7. Monitoring and controlling systems:

Monitoring systems:

A monitoring system can be used to create a record of the condition of a system over a period of time. A monitoring system is used more often to detect when a particular physical property of a system goes outside a desired range.

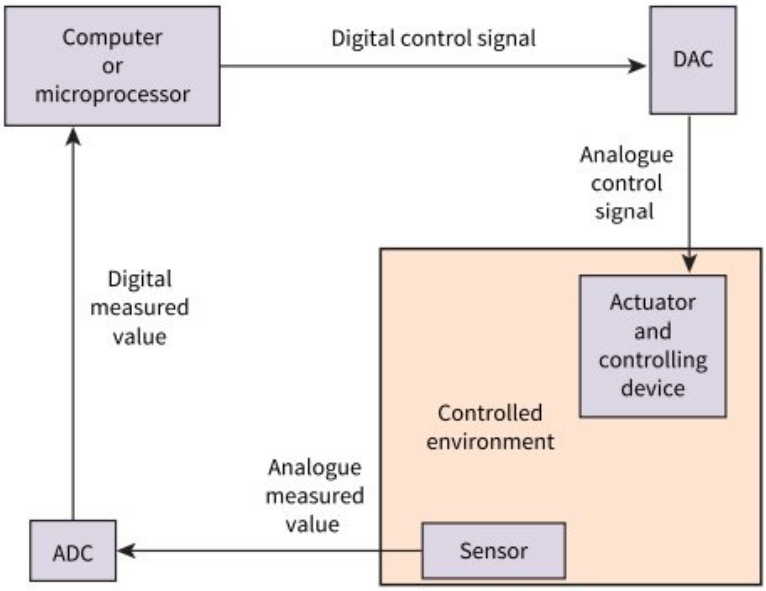
Control system:

A control system has the monitoring activity plus the capability to control a system. The control element of a monitoring and control system needs a device called an actuator.

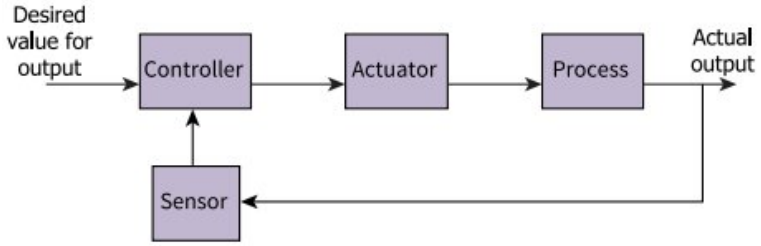
Sensor – A hardware device that measures a property and transmits a value to a controlling computer.

Actuator – A hardware device that receives a signal from a computer and adjusts the setting of a controlling device.

Computer controlled environment:



Closed-loop feedback control system – a type of system where the feedback directly controls the operation.



Bit manipulation to control devices:

The controlling computer or microprocessor has to have a real-time program running continuously. The program can set values for Boolean variables subject to what the sensors detect. Values could be set by assignment statements such as:

IF SensorDifference1 > THEN SensorHighFlag <- TRUE
IF SensorDifference1 < THEN SensorLowFlag <- TRUE
IF SensorDifference2 > THEN SensorHighFlag <- TRUE
IF SensorDifference2 < THEN SensorLowFlag <- TRUE

Another part of the monitoring and control program would then be checking whether any of the four flags were set. The machine code for running such a program could use individual bits to represent each flag. The way flags are could be set and read are illustrated by the following assembly language code fragments. In these code fragments the three least significant bits (position 0, 1 and 2) of the byte are used as flags.

The following illustrates the setting of all bits to zero which might be used when the system is switched on.	
`LDD 0034`	Loads a byte into the accumulator from an address.
`AND #B00000000`	Uses a bitwise AND operation of the contents of the accumulator with the operand to convert each bit to 0.
`STO 0034`	Stores the altered byte in the original address.
The following illustrates the toggling of the value for one bit. This changes the value of the flag it represents. It might be needed because a problem has been encountered or alternatively because a problem has been solved.	
`LDD 0034`	Loads a byte into the accumulator from an address.
`XOR #B00000001`	Uses a bitwise XOR operation of the contents of the accumulator with the operand to toggle the value of the bit stored in position 0.
`STO 0034`	Stores the altered byte in the original address.
The following illustrates the setting of a bit to have value 1 irrespective of its existing value. This would be a simple way of just reporting a condition repetitively.	
`LDD 0034`	Loads a byte into the accumulator from an address.
`OR #B00000100`	Uses a bitwise OR operation of the contents of the accumulator with the operand to set the flag represented by the bit in position 2. All other bit positions remain unchanged.
`STO 0034`	Stores the altered byte in the original address.
The following illustrates setting all bits to zero except one bit which is of interest. Following this operation, a comparison can be made with a binary value to check if the bit is set. In this example the value would be compared to the binary equivalent of denary 2.	
`LDD 0034`	Loads a byte into the accumulator from an address.
`AND #B00000010`	Uses a bitwise AND operation of the contents of the accumulator with the operand to leave the value in position 1 unchanged but to convert every other bit to 0.
`STO 0034`	Stores the altered byte in the original address.

8. System software

System software:

Operating system – A software platform that provides facilities for programs to be run which are of benefit to a user.

Utility system – it is a program it is a program that the user or the operating system can decide to run when needed.

Operating system activities:

What an operating system generally does is to provide an environment where programs can be run that are benefit to a user.

User system interface – This is needed to allow the user to get the software and hardware to do something useful. An operating system should provide the following at the least:

- Command line interface
- Graphical user interface

Program-hardware interface – The operating system must provide the mechanism for running the development program.

Resource management – The resource management provided by the operating system aims to achieve the best possible efficiency in computer system. The two most important aspects of this are:

- Scheduling of process
- Resolution of conflicts when two processes require the same resource

Memory management – There are three important aspects of memory management:

- Memory protection makes sure that different programs don't try to use the same memory location.
- The memory organisation scheme makes sure to achieve the best usage of limited memory.
- Memory usage optimisation involves decisions about which processes should be in main memory at any one time and where they are stored in the memory.

Device management – The management of external devices require:

- Installation of the appropriate device driver software
- Control of usage by process

File management – Three major features are the provision of:

- File naming conventions
- Directory (folder) structures
- Access control mechanisms

Security management – For the operating system that many aspects of security management include:

- Provision for recovery when data is lost
- Prevention of intrusion
- Ensuring data privacy

Error detection and recovery – The operating system should have the capability to interrupt a running process and provide error diagnostics where appropriate. In extreme cases, the operating system needs to be able to shut down the system in an organised fashion without loss of data.

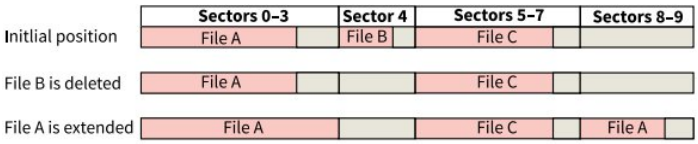
Utility software:

A utility program can be provided by the operating system or it can be installed separately. It is a program that is not executed as part of the normal running of the operating system. Instead it is a program it is a program that the user or the operating system can decide to run when needed.

Disk Formatter – A disk formatter will typically carry out the following tasks:

- Removing existing data from a disk that has been used previously
- Setting up the file system on the disk, based on a table of contents that allows a file recognised by the operating system to be associated with a specific physical part of the disk.
- Partitioning the disk into logical drives if this is required.

Hard defragmentation Software – A hard disk defragmenter is a utility that helps to improve the performance of your hard drive by organizing its data. It does this by moving fragmented files, which are scattered across the hard drive, closer together. This makes it faster for your computer to access the data it needs.



Backup software – You can do a manual backup but it is easier to use a backup utility program that will:

- Establish a schedule for backups
- Only create a new backup file when these has been no change

File compression – This is used to minimise hard disk storage requirements

Virus checker – This checks for viruses.

Program libraries:

A program library contains pre-written subroutines that programmers can use to save time. These subroutines are often tested and reliable. When a program is compiled, it must be linked with any used subroutines.

Including subroutines directly in the executable increases file size and memory usage. An alternative is using Dynamic Link Libraries (DLLs), which store routines separately in memory. DLLs save storage and memory, and can be updated without affecting the main program. However, if a DLL becomes corrupted or buggy, it can cause the program to fail, and troubleshooting can be difficult.

Language translators:

Compiler	Interpreter	Assembler
Translates a high-level language to machine code	Executes a high-level language one statement at a time	Translates low-level assembly language into machine code.
An executable file produced	No executable file produced	An executable file produced.
A single high-level language statement may correspond to multiple machine code instructions.		Each statement in assembly language is translated into one machine code instruction.
Compiler not required to run compiled programs	Interpreted programs cannot be run without the interpreter	Assembler not required to run assembled program
Usually used while distributing program	Used during production stage	Assembled programs are released for general use

Compilers vs interpreters:

Compiler	Interpreter
Translates a high-level language program to machine code.	Translates and executes a high-level language program, line-by-line.
Creates a .exe file which can be easily distributed.	No .exe file created.
Once compiled, .exe file does not need to be compiled again, resulting in faster execution.	Execution very slow – translated each time program run.
Reports all errors at the end of compilation: difficult to locate errors∴ development process long.	Debugging easier/faster, since it stops translating when it reaches an error. This allows real time error correction.
Only be produced when all errors are fixed.	Can run program any time, even before code finished.
Used when development is completed.	Used during development.

Two-step translation:

Some high-level programming languages, such as Java, require a two-step translation process, involving both compilation and interpretation.

Partially compiled programs can be used on different platforms as they are interpreted when run.

Code is optimized for the CPU as machine code is generated at run time

Two-step translation in Java:

- Compilation – Java source code is first converted into an intermediate form called bytecode by the Java compiler.
- Interpretation – The Java Virtual Machine (JVM) then interprets the bytecode, translating it into machine code for execution on the target system.

Features found in a typical integrated development environment (IDE):

Pretty Printing – Automatic indentation and color-coding of keywords

Context-sensitive prompts – Shows keyword you may want to use

Dynamic syntax checks – When a line has been typed, some editors preform syntax checks and alert the programmer to errors

Debugging – Finding and correcting errors, often called bugs

9. Security, privacy and data integrity

Definitions of data integrity, privacy and security:

Data integrity – A requirement for data to be accurate and up to date

Data privacy – A requirement fir data to be available only to authorised users

Data security – A requirement for data to be available for use when needed, ensures that inly authorised users have access to the system and data can be recovered if lost or corrupted

Threats to the security of a computer system and of the data stored in it:

Malware – Malicious software that has the intention of causing harm to a system or its contents.

- Trojan horse – Replaces all or part of a previously useful program
- Spyware – Collects information and transmits it to another system
- Bot Takes control of another computer and uses it to launch attacks
- Phishing – Sending an email or electronic message from an apparently legitimate source requesting confidential information.
- Pharming – Setting up a bogus website which appears to be a legitimate site.
- Keylogger – Recording keyboard usage by the legitimate user of the system.

System vulnerability arising from user activity – Many system vulnerabilities are associated directly with the activities of legitimate user of a system:

- The use of weak passwords, this makes it easy for the hacker to guess the password.
- A user not recognising a phishing or pharming attack.

A legitimate user might introduce a malware deliberately or by accident. This is done by:

- Attaching a portable storage device
- Opening an email attachment
- Accessing a website
- Downloading a file from the Internet

Vulnerability arising from within the system itself – System themselves often have security weakness. Here are some examples:

- Lack of good security
- Commonly used application packages allow micro viruses to spread
- Buffer overflow

Security measures for protecting computer systems:

Disaster recovery – If there was to be a system shutdown at the very least the service should be back up and running after a very short time if it is in a continuity of operation.

Safe system update – The company service should never be shut down so when they are doing an update they have to have the original system and its replacement running in parallel for a period to ensure continuity of service.

User authentication – For any account or PC there should always be a way to authenticate if the user trying to login is the correct user.

Good practice – Don't leave your computer on unattended so that someone else can access it in person or connect a portable storage device that may contain malware.

Firewall – Hardware or software that monitors and controls network traffic.

Digital signature – The sender of an email attaches a digital signature to the email to prove the identity of the sender.

Anti-virus software and intrusion detection – To continuously check for malware

Security measures for protecting data:

Every system should have a backup procedure to recover lost data. The system administrator decides on the details of the procedure. The traditional procedure is quite simple:

- A full backup is made at regular intervals, perhaps weekly
- At least two generations of full backup are kept in storage
- Incremental backups are made on a daily basis

For extra security the backup disks or tapes are stored away from the system.

Authorisation – Definition of a user's access rights to system components

Once a user has logged in, they have been authorised to use a computers system but not necessarily all of it. The system admin may identify different categories of user with different need with respect to the data they are allowed to see and use.

Encrypting data – If your data is hacked it cannot be read by if it is encrypted

Data validation and verification:

Data integrity can never be guaranteed, but the chances are improved if appropriate measures are taken when data originally enters a system or when it is transmitted from one system to another.

Validation – A check that data entered is of the correct type and format; not guarantee that data is accurate

Data validation is implemented by software associated with a data entry interface. There are a number of different types of check that can be made. Typical examples are:

- Presence check – to ensure that an entry field is not left blank
- Format check – for example a date has to be dd/mm/yyyy
- Length check – for example with a telephone number a range check, for example the month in a date must not exceed 12
- Limit check – for example a maximum number of years for a person's age
- Type check – for example only a numeric value for the month in a date
- Existence check – for example that a file exists with the filename referred to in the data entry

Verification – Confirmation of data received by a system

Check digit – Is a value previously calculated from the data that will be entered at some point, for example, the barcode number or an ISBN number. This number stored with the data. When the data is entered, for example, the barcode is scanned, the check digit is recalculated from the data entered. If the previous calculated check digit and stored check digit match the data is correct. If they don't match the data is wrong.

Verification during data transfer – It is possible for data to be corrupted during transmission. Often, this happens when a singular bit is flipped from 1 to 0 or from 0 to 1. Here are some methods:

- Parity check – Can use an odd or even check method. Each byte has 7 bits and 1 extra bit called a parity bit. The parity bit is normally the first or last bit of data in the byte
- Check sum – A calculation is performed on the data sent to create a check sum value and the receiver will do the same calculation and see if the received value is the same.

10. Ethics and Ownership

Ethics:

A system of moral principles that guide behaviour.

In Computer Science these are principles guiding responsible use and development of technology, focusing on privacy, security, fairness, and the impact on society

By joining professional ethical bodies like BCS (British Computer Society) and IEEE (Institute of Electrical and Electronic Engineers) can help computing professionals be ethically guided through their codes of conduct

The code of conduct outlines responsibilities in acting in the best interest of the public, clients, employers, product, judgement, management, profession, colleague and one's self.

Ownership:

Refers to having legal rights and full control over a specific piece or set of data, allowing the holder to make decisions regarding its use, access, and management.

Copyright: Legal right granted to creators of original works, such as music, art, and software, giving them exclusive control over how their work is used, distributed, and reproduced.

Copyright legislation: Laws that grant creators exclusive rights to their original works, protecting them from unauthorized use, reproduction, and distribution.

Types of Software Licensing:

Free Software Foundation – Allows users to run, modify, and distribute the software without restrictions. E.g. Linux

Freeware – Software that is free**.** User cannot view, modify or redistribute the software as source code unavailable. E.g. Zoom and Google docs

The Open Source Initiative – Grants users the rights to inspect, modify, and distribute the software freely**.** Source Code included with software. E.g. Chromium (Software which sets the basis of popular browser)

Shareware – Distributed for free with option to purchase a license for continued use. Often includes limitations or trial periods**.** E.g. Adobe Photoshop

Commercial software – Proprietary software sold for profit, typically with restrictions on how it can be used, modified, or distributed**.** Program code can't be edited. Often come with a license agreement specifying usage rights. E.g. windows, paid video games

Advantage: Support available, less bugs, Software upgrade available, usually compatible with other software.

Artificial Intelligence (AI):

AI is a branch of computer science dealing with the simulation of intelligent behaviours by computers. AI Can learn, decide and act autonomously.

Characteristics:

Collection of data and the rules for using that data: Ai needs huge data sets to operate and perform tasks. Data is collected from various sources like text, image, sensors and human interactions. These data is then processed using a rule that enables the system to form decisions

Ability to reason: AI can make decisions, solve problems, and draw conclusions based on data patterns.

Ability to learn and adapt: AI learns from past experiences, adapting its responses over time to improve performance in changing environments.

Applications:

- Easily process large data and make statistics or analysis
- Can be used in robotics and machines
- Can generate art, music, writing
- Can be used in chat bots or virtual assistance
- Self-driving car or autonomous machines

Impacts:

Social:

- Risk of losing jobs
- Increased leisure time
- Better healthcare

Economic:

- Lower manufacturing cost
- Higher efficiency and accuracy
- Easily Scalable

Environmental:

- Operation of AI may contribute to a carbon footprint
- Traffic and Noise pollution can be reduced drastically
- May lead to increased electronic waste
- Better weather forecasting will lead to better decision making
- Better Agriculture, More production

11. Databases

Limitations of a File-Based Approach:

Data Redundancy & Inconsistency – Data may be duplicated and stored inconsistently, leading to integrity issues.

No Validation Mechanism – Input errors can't be caught automatically.

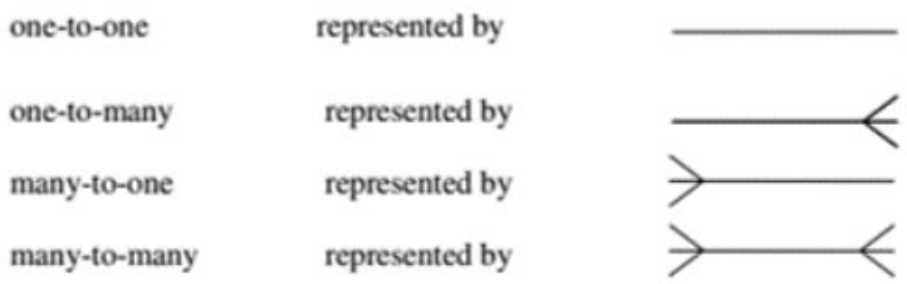
Difficult Data Management – Updating or deleting specific pieces of data is complex and error-prone in flat files.

Relational Database Design:

Relational database – organizes data into tables (relations) with rows (tuples) and columns (attributes)

Primary Key – Uniquely identifies each record

Foreign Key – References a primary key in another table, creating relationships between tables



Entity–Relationship (E–R) Modelling:

A **top-down** design approach

Entities – Represent real-world objects (e.g., Student, Booking)

Relationships – Describe associations between entities

Attributes are associated with entities

Normalization:

A **bottom-up** technique to eliminate redundancy:

- 1NF (First Normal Form) – No repeating groups
- 2NF (Second Normal Form) – No partial dependencies

- 3NF (Third Normal Form) – No transitive dependencies

Improves consistency, reduces duplication, and organizes data logically

Database Architecture (ANSI 3-level):

Internal Level – How data is stored physically (known only by DBMS).

Conceptual Level – Logical structure seen by the DBA.

External Level – Views for users/programs—controls what parts of data they access.

Facilities Provided by a DBMS:

Data Dictionary – Stores metadata (data about data).

Indexing – Improves search performance.

Backup & Recovery – Ensures data integrity and durability.

Query Processor – Handles SQL queries.

User Access Control – Manages who can see or edit data.

Structured query language (SQL):

Industry standard methods for building/modifying databases:

- DBMS uses DDL to create, modify and remove data structures that form a relational database
- DDL statements written as scripts with syntax similar to a computer program
- DBMS uses DML to add, modify, delete and retrieve data STORED in a relational database
- DML statements also written as script with syntax similar to a computer program

SQL (DDL) commands:

```
CREATE DATABASE //creates a database
CREATE TABLE //creates a table
ALTER TABLE //changes definition of a table
PRIMARY KEY //adds primary key to table
FOREIGN KEY ... REFERENCES ... //adds foreign key to table
```

SQL (DDL) data types for attributes:

```
CHARACTER //fixed length text
VARCHAR(n) //variable length text
BOOLEAN //true or false, uses 1 and 0
INTEGER //whole number
REAL //number with decimal places
DATE //formatted as YYYY-MM-DD
TIME //formatted as HH:MM:SS
```

SQL (DDL) query commands:

```
SELECT FROM //fetches data from database, queries always begin with
select
WHERE //includes only rows that match given condition
ORDER BY //sorts results from a query in a column either
alphabetically or numerically
GROUP BY // arranges data into groups
INNER JOIN //combined rows from different tables if the join condition
is true
SUM //sum of all values in column
COUNT //counts number of rows where the column is not NULL
AVG //average for a column with a numeric data type
```

SQL (DDL) maintenance commands:

```
INSERT INTO //adds new row
DELETE FROM //remove row
UPDATE //edit row
```

12. Algorithm and design and problem-solving

Computational thinking:

Abstraction – This is the action of filtering out information that is not necessary to solve a problem

Decomposition – Breaking down problems into sub-problems

Data modelling – This involves analysing and organising data

Pattern recognition – Looking for patterns or common solutions to common problems and using these to complete tasks in a more efficient and effective way

Algorithm design – Using step by step instructions to solve a problem

- Algorithm – A sequence of defined steps that can be carried out to perform a task.

Expressing algorithms:

Structured English – A subset of the English language that consists of command statements used to describe an algorithm

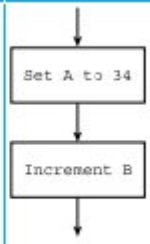
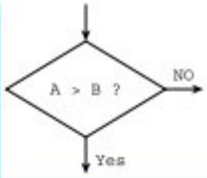
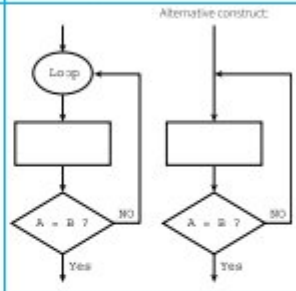
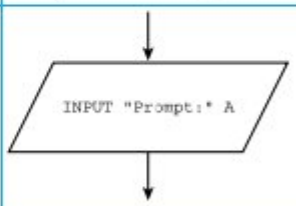
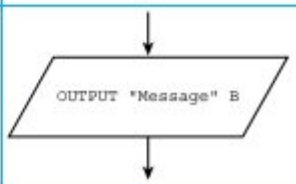
Pseudocode – A way of using keywords and identifiers to describe an algorithm without following the syntax of a particular programming language

Flowchart – Shapes linked together to represent the sequential steps of an algorithm

An algorithm consists of certain steps. Under certain conditions we may wish not to perform some steps or repeat a number of steps. We use four basic types of construct.

- Assignment – A value is given a name (identifier) or the value associated with a given identifier is changed
- Sequence – A number of steps are preformed, one after the other
- Selection – Under certain conditions some steps are preformed, otherwise different (or no) steps are performed
- Repetition – A sequence of steps is performed a number of times. This is also known as iteration or looping

Many problems we try to solve with a computer involve data. The solution involves inputting data to the computer, processing the data and outputting the results

	Structured English	Pseudocode	Flowchart
Assignment and Sequence	SET A TO 34 INCREMENT B	A ← 34 B ← B + 1	
Selection	IF A IS GREATER THAN B THEN ... ELSE ...	IF A > B THEN ... ELSE ... ENDIF	
Repetition	REPEAT UNTIL A IS EQUAL TO B ...	REPEAT ... UNTIL A = B	
Input	INPUT A	INPUT "Prompt: " A	
Output	OUTPUT "Message" OUTPUT B	OUTPUT "Message", B	

Variables:

Variable – A storage location for a data value that has in identifier

Identifier table – A table listing the variables required for a solution, with explanations and data types

Logic statements:

Simple Condition

```
IF A < B
    THEN
        <statement (s)>
    ELSE
        <statement (s)>
ENDIF
```

A condition consists of at least one logic proposition.

Conditions are either TRUE or FLASE. In pseudocode

Nested IF statements – Conditional statements within conditional statements

Loops:

If we use loops we can avoid writing the same lines of code over and over again.

Rogue value – A value used to terminate a sequence of values

Nested loop – Loop containing another loop

Stepwise refinement:

Stepwise refinement – Breaking down the steps of an outline solution into smaller and smaller steps until they are solved easily

Modules:

Another method of developing a solution is to decompose the problem into sub-tasks. Each sub-task is considered a 'module' that is refined separately. Modules are procedures and functions.

Procedure – A sequence of steps that is given an identifier and can be called to perform a sub-task

Function – A sequence of steps that is given an identifiers and returns a single value; function call is part of an expression

The difference between them is that functions return a value to the program where procedures perform a specific task.

Local variable – A variable that id accessible only within the module in which it is declared

Global variable – A variable that is accessible from all modules

13. Data types and structures

Data types:

Primitive data types – These data types are variables that can be defined by commands build into the programming language.

Further data types – This is a data type that is created by the programmer

INTEGER	A signed whole number
REAL	A signed number with a decimal point
CHAR	A single character
STRING	A sequence of zero or more characters
BOOLEAN	The logical values TRUE and FALSE
DATE	A date consisting of day, month and year, sometimes including a time in hours, minutes and seconds

The record type:

Sometimes variables of different data types are a logical group, such as data about a person: name, date of birth, height, number of siblings. Each of these have their own data type.

We can declare a record type to suit our purpose. The record type is known as a user-defined type, because the programmer can decide which variables (fields) to include as a record.

A record types is declared as:

```
TYPE <TypeIdentifier>
    DECLARE <field identifier> : <data type>
    .
    .
ENDTYPE
```

We can now declare a variable of this record type:

```
DECLARE <variable identifier> : <record type>
```

And then access an individual field using the dot notation:

```
<variable identifier>.<field identifier>
```

Using the example above we can declare a Person record type:

```
TYPE PersonType
    Name : STRING
    DateOfBirth : DATE
    Height : REAL
    NumberOfSiblings : INTEGER
    IsFullTimeStudent : BOOLEAN
ENDTYPE
```

To declare a variable of this type we write:

```
DECLARE Person : PersonType
```

And now we can assign a value to a field of this Person record:

```
Person.Name ← "Fred"
Person.NumberOfSiblings ← 3
Person.IsFullTimeStudent ← TRUE
```

To output a field of a record:

```
OUTPUT Person.Name
```

Arrays:

An array is an ordered set of data items, usually of the same type, grouped together using a single identifier.

Array index – Row or columns number of an individual array element

Upper bound – The highest number index of an array dimension

Lower bound – The smallest number index of an array dimension

One-dimensional array:

A 1D array is like a numbered list of items.

In pseudocode, a 1D array is written as:

DECLARE <arrayIdentifiers>: ARRAY [<lowerBound> : < upperBound>] OF <datatype>

Here is a pseudocode example:

DECLARE List1: ARRAY [0:5] OF STRING There are 6 elements in this list

Accessing 1D arrays – A specific element in an array is accessed using an index value. In pseudocode this is written as:

<ArrayIdentifier>[x]

Linear search – Checking each element of an array in turn for a required value

Bubble sort – A sort method where adjacent pairs are compared and swapped

Two dimensional arrays:

A 2D array is like a table and you want to refer to an individual element of the table.

In pseudocode, a 2D array is written as:

DECLARE <arrayIdentifiers>: ARRAY [<LBound1> : < UBound1>, <LBound2> : < UBound2>] OF <datatype>

Here is a pseudocode example:

DECLARE List1: ARRAY [0:5, 0:7] OF INTEGER

Accessing 2D arrays – A specific element in a table is accessed using an index pair. In pseudocode this is written as:

<ArrayIdentifier>[x, y]

Text files:

To store data permanently files can be used. Data held in an array will be lost when the program is stopped. You can save the data to a file and read it back in when your program requires it.

A text file consists of a sequence of characters formatted into lines. Each line is terminated by an end-of-line marker. The text file is terminated by an end-of-file marker.

Writing to a text file – Writing to a text file usually means creating a text file

```
OPENFILE <filename> FOR WRITE       // open the file for writing
WRITEFILE <filename>, <stringValue>  // write a line of text to the file
CLOSEFILE <filename>               // close file
```

Reading from a text file – An existing file can be read by a program

```
OPENFILE <filename> FOR READ          // open file for reading
READFILE <filename>, <stringVariable>   // read a line of text from the file
CLOSEFILE <filename>                 // close file
```

Appending to a text file – Sometimes we may wish to add data to an existing file rather than create a new file. This can be done in Append mode it adds new data to the end of an existing file

```
OPENFILE <filename> FOR APPEND     // open file for append
WRITEFILE <filename>, <stringValue> // write a line of text to the file
CLOSEFILE <filename>              // close file
```

The end of file (EOF) marker – If we want to read a file from beginning to end, we can use a conditional loop. Text files contain a special marker at the end of the file that we can test for. Testing for this special end-of-file marker is a standard function in many programming languages. Every time this function is called it will test for this marker. The function will return TRUE if the end-of-file marker has been reached.

In pseudocode we call this function EOF(). We can use the construct REPEAT…UNTIL EOF().

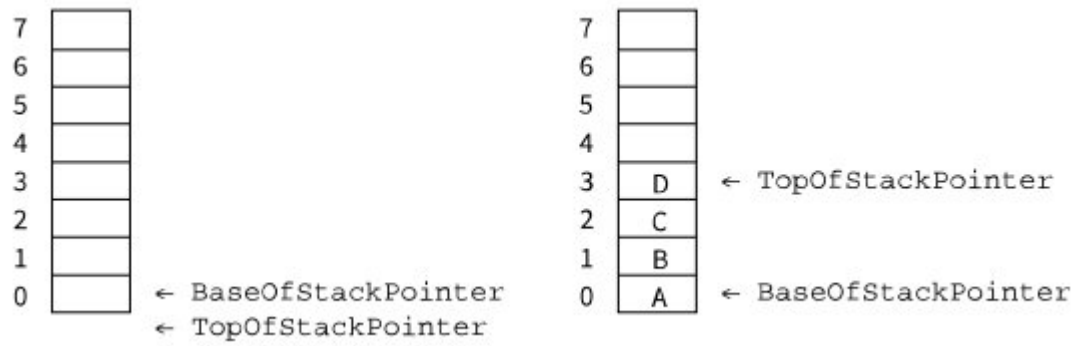
If it possible that the file contains no data, it is better to use the construct WHILE NOT EOF().

Abstract Data Type (ADTs):

Abstract data type – A collection of data with associated operations:

- Create a new instance of the data structure
- Find an element in the data structure
- Insert a new element into the data structure
- Delete an element from the data structure
- Access all elements stored in the data structure in a systematic manner

Stacks:



Top of stack pointer – Always points to the top item in the stack

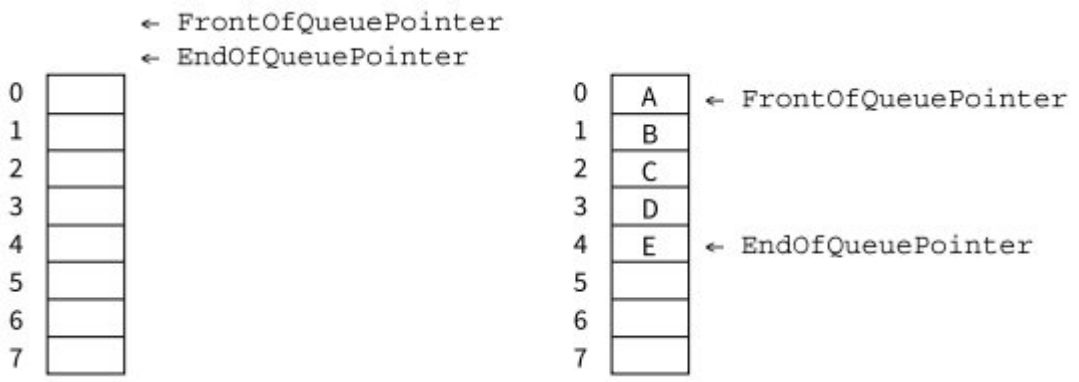
Base of attack pointer – Always points to the first slot in the stack

When an item is placed into the stack it goes to the first available slot. Any items bellow that item cannot be accessed unit it is removed. When there are no items in the stack the Top of stack pointer will have the value of -1.

To implement a stack using a 1D array, we write:

DECLARE Stack: ARRAY[0:7] OF CHAR

Queues:

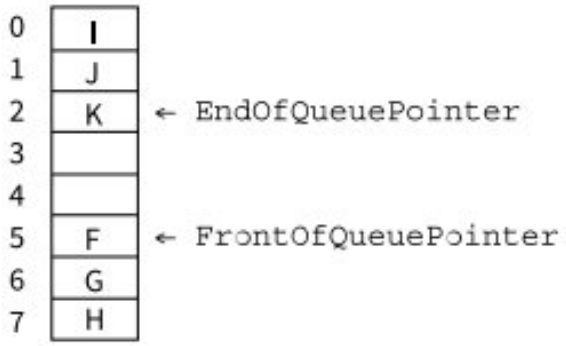


Front of queue pointer – This will hold the spot where the items will leave the queue

End of queue pointer – This will point to the end of the queue

When an item joins the queue it must enter from behind the end of queue pointer and the end of queue pointer will move to the new item at the end. When an item leaves the queue all the items move one forward. When there are no items in the queue both pointers will have the value of -1.

The last method have a lot of moving of data. A more efficient way to make use of the slots is the concept of a 'circular' queue. When the front item leaves the queue instead of everything we can just move the Front pointer one back and when an item joins the queue we can move the end pointer one back.



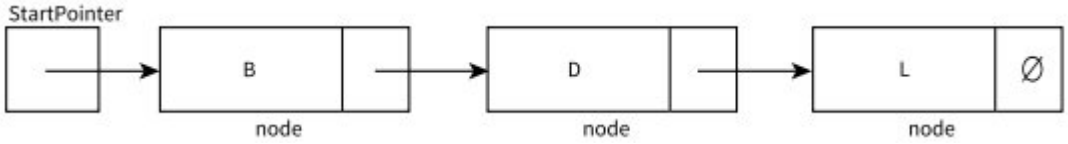
Linked list:

Node – An element of a list

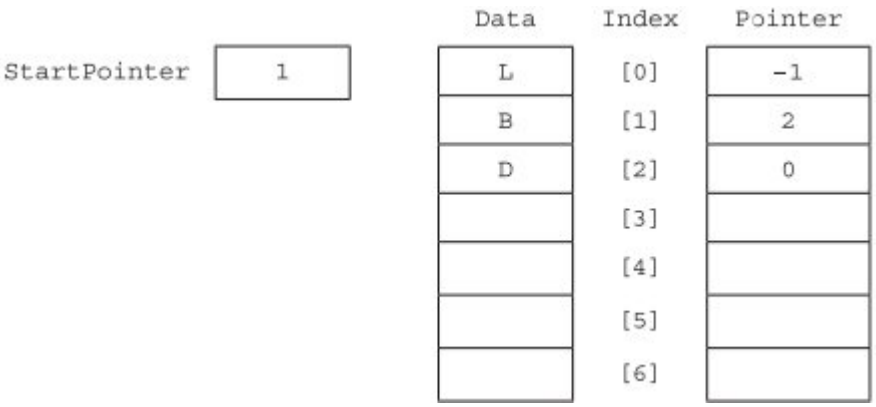
Pointer – A variable that stores the address of the node it points to

Null pointer – A pointer that does not point at anything

Start pointer – A variable that stores the address of the first element of a linked list



This is how you could use this in a 2D array:



14. Programming and data representation

Programming basics:

All that is written here is in pseudocode. The python is in this file:

↵ Programming.pdf

Declaration of variables:

DECALRE <identifier> : <datatype>

Declaration and assignments of constants:

CONSTANT <identifier> = <value>

Assignment of variables:

<identifier> <- <expression>

Arithmetic operators:

Operation	Pseudocode	Python	VB.NET	Java
Addition	+	+	+	+
Subtraction	-	-	-	-
Multiplication	*	*	*	*
Division	/	/	/	/ (when dividing float or double types)
Exponent	^	**	^	No operator available, only method: `Math.pow(n,e)`
Integer division	DIV	//	\	/ (when dividing integer types)
Modulus	MOD	%	Mod	%

Outputting information to the screen:

OUTPUT <string>

OUTPUT <identifier>

Getting input from the user:

INPUT "Enter a number:" A

Data types:

Description of data	Pseudocode	Python	VB.NET	Java
Whole signed numbers	INTEGER	int	Integer (4 bytes)	int (4 bytes)
Signed numbers with a decimal point	REAL	float	Single (4 bytes) Double (8 bytes)	float (4 bytes) double (8 bytes)
A single character	CHAR Use single (') quotation marks to delimit a character	Not available	Char (2 bytes – Unicode)	char (2 bytes – Unicode)
A sequence of characters (a string)	STRING Use double (") quotation marks to delimit a string.	str (stored as ASCII but Unicode strings are also available) Use single ('), double (") or triple (''' or """) quotation marks to delimit a string.	String (2 bytes per character) Use double (") quotation marks to delimit a string.	String (2 bytes per character) Use double (") quotation marks to delimit a string.
Logical values: True (represented as 1) and False (represented as 0)	BOOLEAN	bool possible values: True False	Boolean (2 bytes) possible values: True False	Boolean possible values: true false

Date is not built into python and java

Description of data	Pseudocode	Python	VB.NET	Java
Date value	DATE	Use the datetime class	Date (8 bytes)	Date is a class in Java. To make use of it use: import java.util.Date;

Boolean expressions:

Comparison operators:

Operation	Pseudocode	Python	VB.NET	Java
equal	=	==	=	==
not equal	<>	!=	<>	!=
greater than	>	>	>	>
less than	<	<	<	<
greater than or equal to	>=	>=	>=	>=
less than or equal to	<=	<=	<=	<=

Boolean operators:

Operation	Pseudocode	Python	VB.NET	Java
AND (logical conjunction)	AND	and	And	&&
OR (logical inclusion)	OR	or	Or	||
NOT (logical negation)	NOT	not	Not	!

Selection:

IF…THEN statements:

```
IF <Boolean expression>
    THEN
        <statement>
ENDIF
```

IF…THEN…ELSE statements:

```
IF <Boolean expression>
    THEN
        <statement>
    ELSE
        <statement>
ENDIF
```

Nested IF statements:

```
IF <Boolean expression>
    THEN
        <statements>
    ELSE
        IF <Boolean expression>
            THEN
                <statement>
            ELSE
                <statement>
        ENDIF
ENDIF
```

Case statements:

```
CASE OF <expression>
    <value1>                : <statement>
    <value2>, <value3>      : <statement>
    <value4> TO <value5>    : <statement>
    OTHERWISE <statement>
ENDCASE
```

Eg.

```
CASE OF Grade
  "A"       : OUTPUT "Top grade"
  "F", "U" : OUTPUT "Fail"
  "B".."E" : OUTPUT "Pass"
OTHERWISE OUTPUT "Invalid grade"
ENDCASE
```

Iteration:

Count-controlled (FOR) loops - In pseudocode, count-controlled loop is written as:

```
FOR <control variable> ← s TO e STEP i // STEP is optional
    <statement(s)>
NEXT <control variable>
```

The control variable starts with value s, increments by value I each time round the loop and finished when the control variable reaches the value e

In python a count-controlled loop is written as:

```
for <control variable> in range (s, e, i) :
        <statement>
```

Post-conditional loop – A post-conditional loop makes sure that the loop runs at least once. In pseudocode a post-conditional loop is written as:

```
REPEAT
    <statement(s)>
UNTIL <condition>
```

There is no post-conditional loop option in python

Pre-conditional loop – A pre-conditional loop evaluates the condition before the statements within the loop are executed. In pseudocode a pre-conditional loop is written as:

```
WHILE <condition> DO
    <statement(s)>
ENDWHILE
```

In python a pre-conditional loop is written as:

```
while <condition> :
        <statement(s)>
```

Built in functions:

String manipulation functions:

Description	Pseudocode	Python	VB.NET	Java
Access a single character using its position P in a string `ThisString`	`ThisString[P]` Counts from 1	`ThisString[P]` Counts from 0	`ThisString(P)` Counts from 0	`ThisString.charAt(P)` Counts from 0
Returns the character whose ASCII value is i	`CHR(i : INTEGER) RETURNS CHAR`	`chr(i)`	`Chr(i)`	`(char) i;`
Returns the ASCII value of character ch	`ASC(ch) RETURNS INTEGER`	`ord(ch)`	`Asc(ch)`	`(int) ch;`
Returns the integer value representing the length of string S	`LENGTH(S : STRING) RETURNS INTEGER`	`len(S)`	`len(S)`	`S.length();`
Returns leftmost L characters from S	`LEFT(S : STRING, L : INTEGER) RETURNS STRING`	`S[0:L]` See the next section, on slicing	`Left(S, L)`	`S.subString (0, L)`
Returns rightmost L characters from S	`RIGHT(S: STRING, L : INTEGER) RETURNS STRING`	`S[-L:]` See the next section, on slicing	`Right(S, L)`	`S.subString (S.length() – L)`
Returns a string of length L starting at position P from S	`MID(S : STRING, P : INTEGER, L : INTEGER) RETURNS STRING`	`S[P : P + L]` See the next section, on slicing	`mid(S, P, L)`	`S.subString(P, P + L)`
Returns the character value representing the lower case equivalent of Ch	`LCASE(Ch : CHAR) RETURNS CHAR`	`Ch.lower()`	`LCase(Ch)`	`Character.toLowerCase(ch)`
Returns the character value representing the upper case equivalent of Ch	`UCASE(Ch : CHAR) RETURNS CHAR`	`Ch.upper()`	`UCase(Ch)`	`Character.toUpperCase(ch)`
Returns a string formed by converting all alphabetic characters of S to upper case	`TO_UPPER(S : STRING) RETURNS STRING`	`S.upper()`	`S.ToUpper`	`S.toUpperCase()`
Returns a string formed by converting all alphabetic characters of S to lower	`TO_LOWER(S : STRING) RETURNS STRING`	`S.lower()`	`S.ToLower`	`S.toLowerCase()`
Concatenate (join) two strings	`S1 & S2`	`s = S1 + S2`	`s = S1 + S2` `s = S1 & S2`	`s = S1 + S2;`

Truncating numbers – Instead of rounding you may just want the whole number part of a number:

Pseudocode	`INT(x : REAL) RETURNS INTEGER`	Returns the integer part of `x`.
Python	`int(x)`	If `x` is a floating-point number, the conversion truncates towards zero.
VB.NET	`Math.Truncate(x)`	The whole number part of the real number `x` is returned.
Java	`(int) x;`	Casts the number as an integer.

Converting a string to a number – If you want to convert a string to a number in python for example the string "5" stored in the variable S:

```
int (S)
```

If there is a decimal in the number that is stored int the string then all you do is:

```
float (S)
```

In pseudocode this is how you do it:

```
STRING_TO_NUMBER (x: STRING) RETURNS REAL
```

Random number generator:

```
randint (1, 6)
```

Procedures and functions:

You have already learned about procedures and functions but this is how you create them:

```
def <identifier> () :
    <statements>
```

In order to change this into a function all you must do is return a value:

```
def <identifier> () :
    <statements>
    return <value>
```

Return value – The value replacing the function call used in the expression

Passing parameters to subroutines:

In the brackets you put a variable that is then passed to the subroutine. When calling the subroutine you can put in a value that will then be passed into the subroutine as the variable.

```
def SumRange(FirstValue, LastValue):
    Sum = 0
    for ThisValue in range(FirstValue, LastValue + 1):
        Sum = Sum + ThisValue
    return Sum

# ********* main program starts here **************
NewNumber = SumRange(1, 5)
print(NewNumber)
```

Arrays:

Declaring a 1D array is pseudocode:

DECLARE <arrayIdentifier>: ARRAY [<lowerBound>: <upper Bound>] OF <datatype>

Python examples:

Python	`List1 = []`
	`List1.append("Fred")`
	`List1.append("Jack")`
	`List1.append("Ali")`
		As there are no declarations, the only way to generate a list is to initialise one. You can append elements to an existing list.

	`List2 = [0, 0, 0, 0, 0, 0]`	You can enclose the elements in `[ ]`.
	`List3 = [0 for i in range(100)]`	You can use a loop.
	`AList = [""] * 26`	You can provide an initial value, multiplied by number of elements required.

Accessing 1D arrays in pseudocode and python:

<ArrayIdentifier>[x]

Creating 2D arrays in pseudocode:

DECLARE <arrayIdentifier>: ARRAY [<lowerBound>: <upper Bound>], [<lowerBound>: <upper Bound>] OF <datatype>

Python examples:

Python	`Board = [[0, 0, 0, 0, 0, 0, 0],`
	`[0, 0, 0, 0, 0, 0, 0],`
	`[0, 0, 0, 0, 0, 0, 0],`
	`[0, 0, 0, 0, 0, 0, 0],`
	`[0, 0, 0, 0, 0, 0, 0],`
	`[0, 0, 0, 0, 0, 0, 0]]`
	`Board = [[0 for i in range(7)]`
	`for j in range(6)]`
	`Board = [[0] * 7] * 6`
		2D lists can be initialised in a similar way to 1D lists. Remember that elements are numbered from 0. These are alternative ways of initialising a 6 × 7 list. The rows are numbered 0 to 5 and the columns 0 to 6. The upper value of the range is not included.

Accessing 2D arrays in pseudocode:

<ArrayIdentifier>[x, y]

Accessing 2D arrays in python:

<ArrayIdentifier> [2] [3] = 0 Elements are numbered from 0 in python so this gives access to the 4th element in the 3rd element

Text files:

Writing to a text file:

```
OPENFILE <filename> FOR WRITE       // open the file for writing
WRITEFILE <filename>, <stringValue> // write a line of text to the file
CLOSEFILE <filename>                // close file
```

Python	`FileHandle = open("SampleFile.TXT", "w")`
	`FileHandle.write(LineOfText)`
	`FileHandle.close()`
		You specify the filename and mode ('w' for write) when you call the `open` function. The line of text to be written to the file must contain the newline character "\n" to move to the next line of the text file.

Reading from a text file:

```
OPENFILE <filename> FOR READ           // open file for reading
READFILE <filename>, <stringVariable>  // read a line of text from the file
CLOSEFILE <filename>                   // close file
```

Python	`FileHandle = open("SampleFile.TXT", "r")`
	`LineOfText = FileHandle.readline()`
	`FileHandle.close ()`
		You specify the filename and mode ('r' for read) when you call the `open` function.

Appending to a text file:

```
OPENFILE <filename> FOR APPEND      // open file for append
WRITEFILE <filename>, <stringValue> // write a line of text to the file
CLOSEFILE <filename>                // close file
```

Python	`FileHandle = open("SampleFile.TXT", "a")`
	`FileHandle.write(LineOfText)`
	`FileHandle.close()`
		You specify the filename and mode ('a' for append) when you call the `open` function.

The end-of-file (EOF) marker:

```
OPENFILE "Test.txt" FOR READ
WHILE NOT EOF("Test.txt") DO
    READFILE "Test.txt", TextString
    OUTPUT TextString
ENDWHILE
CLOSEFILE "Test.txt"
```

Python	`FileHandle = open("Test.txt", "r")`
	`LineOfText = FileHandle.readline()`
	`while len(LineOfText) > 0:`
	`  LineOfText = FileHandle.readline()`
	`  print(LineOfText)`
	`FileHandle.close()`
		There is no explicit EOF function. However, when a line of text has been read that only consists of the end-of-file marker, the line of text is of length 0.

15. Software development

Stages in the program development life cycle:

Analyse problem – Define problem, record program specifications and recognize inputs, process, output & UI

Design program – Develop logic plan, write algorithm in e.g. pseudocode or flowchart and test solution

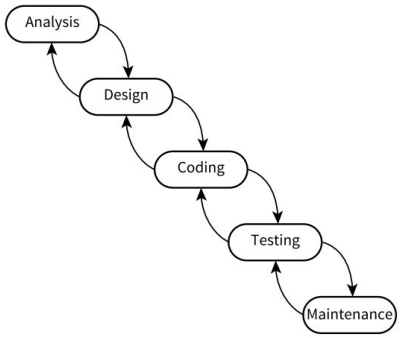
Code program – Translate algorithm into high level language with comments/remarks and produce user interface with executable processes

Test and debug program – Test program using test data, find and correct any errors and ensure results are correct

Maintain program – Provide education and support to end-user, correct any bugs and modify if user requests

The program development life cycle:

Waterfall model – A classical model, used to create a system with a linear approach, from one stage to another



Iterative model – An initial representation starts with a small subset, which becomes more complex over time until the system is complete

Rapid Application Development (RAD) model – A prototyping model, with no (or less) specific planning put into it. More emphasis on development and producing a product-prototype

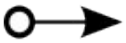
Program design using structure charts:

Structure chats – A graphical representation of the modular structure of a solution

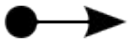
Parameter – A value passed between modules

Process – Represents a programming module e.g. a calculation

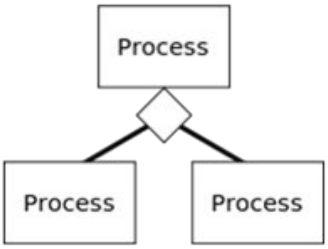
Data couple – Data being passed from module to module that needs to be processed



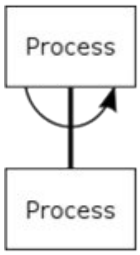
Flag – Check data sent to start or stop a process. E.g. check if data sent in the correct format



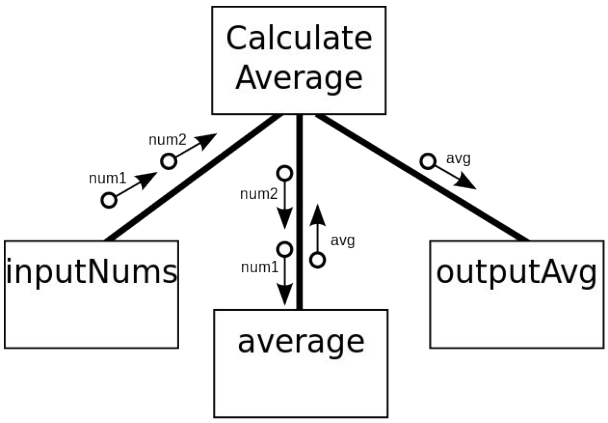
Selection – Condition will be checked and depending on the result, different modules will be executed



Iteration – Implies that module is executed multiple times



Example:



Converting Pseudocode to Structure Chart

- Break the program into main task and sub tasks
- Define main module and submodules
- Each module will be represented in boxes
- Parameters passed by value will go to the left
- Parameters passed by reference and return value goes to right

Program design using state-transition diagrams:

Finite state machine (FSM) – A machine that consists of a fixed set pf possible states with a set of inputs that change the state and a set of possible outputs

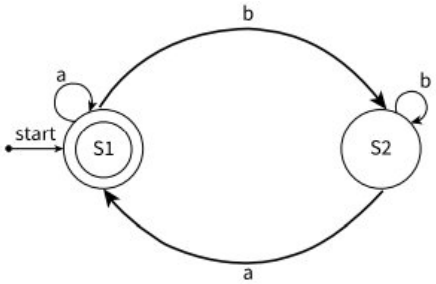
State-transition table – A table that gives information about the states of an FSM

State-transition diagram – A diagram that diagram that describes the behaviour of an FSM

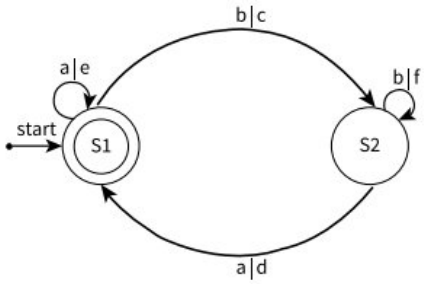
Here is an example:

- If the FSM is in state S1, an input of a causes no change of state
- If the FSM is in state S1, an input of b transforms S1 to S2
- If the FSM is in state S2, an input of b causes no change of state
- If the FSM is in state S2, an input of a transforms S2 to S1

		current state	
		S1	S2
input	a	S1	S1
	b	S2	S2



This is what it would look like with outputs:



If an input causes an output this is shown by a vertical bar. For example, if the current state is S1, an input of b produces output c and transforms the FSM to state S2.

Types of errors:

Syntax errors – An error in which a program statement does not follow the rules of the language

Logic error – An error in the logic of the solution that causes it not to behave as intended

Run-time error – An error that causes program execution to crash or freeze

Testing errors:

Stub testing – When you want to test your program before you have implemented all the facilities you can write a 'stub' for each procedure. The procedure body only contains an output statement to acknowledge that the call was made.

Black-box testing – Comparing expected results with actual results when a program is run

White-box testing – Testing every path through the program code

Dry-run (walk through) – The process of checking the execution of an algorithm or program by recording variable values in a trace table

Integration testing – Individually tested modules are joined into one program and tested to ensure the modules interact correctly

Alpha testing – Testing of software in-house by dedicated testers

Acceptance testing – Testing of software by customers before sign-off

Beta testing – Testing of software by a limited number of chosen users before general release

Test data:

Normal (valid) – Typical data values that should be accepted

Abnormal (erroneous) – Data values that the system should not accept

Boundary (extreme) – Data values that are at a boundary or an extreme end of range of normal data; test data should include values just within the boundary (that is, valid data) and just outside the boundary (that is, invalid data)

Maintenance:

Corrective maintenance – Correcting identified errors

Adaptive maintenance – Amending a program to enhance functionality or in response to specifications changes

Perfective maintenance – Modifying a program to improve performance or maintainability